

План собеседования Middle+ Go разработчика

Оглавление

Пункты оглавления являются внутренними ссылками: нажмите на раздел или вопрос, чтобы перейти к соответствующему месту в PDF.

Цель интервью

Ожидаемый профиль Middle+

Рекомендуемая структура интервью

1. Вводная часть и опыт

- Цели блока
- Основные вопросы
- Сильные сигналы
- Риски

2. Go core и идиоматика

- Темы для проверки
- Вопросы по языку
- Практические мини-вопросы

3. Concurrency и context

- Темы для проверки
- Основные вопросы
- Практический concurrency-сценарий

4. Backend, API и работа с данными

- HTTP/gRPC и API
- Базы данных
- Reliability

5. Практическая задача

- Вариант A: In-memory rate limiter
- Вариант B: Concurrent fetch with timeout
- Вариант C: Transactional service method

6. System design light

- Сценарий
- Что попросить кандидата раскрыть
- Уточняющие вопросы
- Сильный ответ

7. Тестирование и качество

- Вопросы
- Практическое обсуждение

8. Performance и profiling

- Вопросы
- Сильные сигналы

9. Observability и эксплуатация

- Вопросы

10. Security

- Вопросы

11. Code review

- Формат
- Вопросы
- Сильный ответ

12. Поведенческий блок

13. Матрица оценки

Компетенции

Decision guide

14. Красные флаги

15. Вопросы кандидата к интервьюеру

16. Итоговый шаблон заметок интервьюера

17. Подробные вопросы и ответы для интервьюера

17.1 Go core и идиоматика

1. Чем slice отличается от array? Что хранит slice header?
2. Что происходит при append к slice при достаточной и недостаточной capacity?
3. Почему изменение одного slice может неожиданно изменить другой slice?
4. Можно ли читать из nil map? Можно ли писать в nil map?
5. Что происходит при обращении к отсутствующему ключу map?
6. Почему map в Go не потокобезопасна?
7. Когда стоит использовать pointer receiver, а когда value receiver?
8. Чем interface value с nil concrete value отличается от nil interface?
9. Как Go выбирает, реализует ли тип интерфейс?
10. Для чего нужен empty interface или any? Когда его использование ухудшает дизайн?
11. Как работает defer? В каком порядке выполняются несколько defer?
12. Что будет с аргументами defer: вычисляются сразу или в момент выполнения?
13. Когда допустимо использовать panic/recover в backend-сервисе?
14. Как вы обычно оборачиваете и сравниваете ошибки?
15. Чем errors.Is отличается от errors.As?
16. Когда уместны sentinel errors, а когда лучше typed errors?
17. Что такое zero value и почему это важная идея в Go?
18. Как generics изменили подход к utility-коду? Когда не стоит использовать generics?
19. Какие пакеты стандартной библиотеки вы используете чаще всего?
20. Что такое escape analysis и как она может влиять на производительность?
21. Чем stack отличается от heap в Go?
22. Где хранятся локальные переменные функции и всегда ли локальная переменная находится на stack?
23. Почему возврат указателя на локальную переменную безопасен в Go?
24. Как растет stack goroutine и почему это важно для большого количества goroutines?
25. Почему утверждение "указатели всегда быстрее" неверно?
26. Как диагностировать лишние heap allocations?

17.2 Concurrency и context

27. Чем goroutine отличается от OS thread?
28. Что приводит к goroutine leak?
29. Как обнаружить goroutine leak в production?
30. Когда использовать channel, а когда mutex?
31. Что происходит при чтении из закрытого channel?
32. Что происходит при записи в закрытый channel?
33. Для чего нужен select с default?
34. Как реализовать fan-out/fan-in?
35. Как ограничить количество одновременно выполняющихся задач?
36. Как корректно остановить worker pool?
37. Как работает context.Context и что нельзя хранить в context?
38. Почему нельзя игнорировать ctx.Done() в долгих операциях?
39. Что такое data race?
40. Когда использовать sync/atomic вместо mutex?
41. Что такое backpressure?
42. Как сделать graceful shutdown HTTP-сервера?

17.3 Backend, API и данные

43. Как проектировать REST API для нового ресурса?
44. Какие HTTP-коды использовать для типовых ошибок?
45. Где должна жить бизнес-логика: handler, service или repository?
46. Как реализовать idempotency для POST-запроса?
47. Когда выбирать gRPC вместо REST?
48. Что такое транзакция?
49. Какие уровни изоляции транзакций вы знаете?
50. Что такое dirty read, non-repeatable read, phantom read?
51. Как избежать lost update?
52. Как работает optimistic locking?
53. Как выбрать индекс для SQL-запроса?
54. Почему индекс может не использоваться?
55. Что такое N+1 problem и как исправлять?
56. Как проектировать миграции без downtime?
57. Как хранить nullable-поля в Go?
58. Offset vs cursor pagination?

17.4 Reliability и внешние зависимости

- 59. Что такое timeout, retry и circuit breaker?
- 60. Почему retries могут ухудшить инцидент?
- 61. Что такое partial failure?
- 62. Как проектировать взаимодействие с внешним API?
- 17.5 Практические задачи: ожидаемые решения
 - 63. In-memory rate limiter: какой алгоритм выбрать?
 - 64. Concurrent fetch with timeout: как должна выглядеть архитектура?
 - 65. Transactional money transfer: как решить безопасно?
- 17.6 Тестирование
 - 66. Что такое table-driven tests?
 - 67. Как тестировать код, зависящий от времени?
 - 68. Как тестировать HTTP handler?
 - 69. Как тестировать интеграцию с БД?
 - 70. Когда использовать mocks, fakes и testcontainers?
- 17.7 Performance и profiling
 - 71. Как понять, где bottleneck: CPU, память, БД или сеть?
 - 72. Как использовать pprof?
 - 73. Что может привести к высокому GC pressure?
 - 74. Как искать memory leak в Go?
- 17.8 Observability и эксплуатация
 - 75. Какие structured logs нужны HTTP-сервису?
 - 76. Какие RED/USE-метрики вы знаете?
 - 77. Readiness vs liveness?
- 17.9 Security
 - 78. Как хранить пароли пользователей?
 - 79. Authentication vs authorization?
 - 80. Как устроен JWT и какие риски?
 - 81. Как защищаться от SQL injection?
- 17.10 Code review
 - 82. Как ревьюить PR с business logic в handler?
 - 83. Как приоритизировать замечания в code review?
- 17.11 System design: сервис уведомлений
 - 84. Какой базовый дизайн предложить?
 - 85. Как не отправить уведомление дважды?
 - 86. Как гарантировать, что событие не потеряется?
 - 87. Что делать, если SMS-провайдер недоступен 30 минут?
 - 88. Какие метрики нужны сервису уведомлений?
- 17.12 Поведенческий блок: что считать сильным ответом
 - 89. Кандидат не согласился с техническим решением команды. Что слушать в ответе?
 - 90. Кандидат ошибся в production. Что слушать?
 - 91. Как кандидат выбирает между quick fix и refactoring?
 - 92. Как кандидат помогает менее опытному разработчику?
- 17.13 Расширенный блок по базам данных
 - 93. Что происходит внутри БД при выполнении SQL-запроса?
 - 94. Чем EXPLAIN отличается от EXPLAIN ANALYZE?
 - 95. Какие поля в EXPLAIN ANALYZE вы смотрите первыми?
 - 96. Что такое selectivity и cardinality индекса?
 - 97. Как работает composite index и почему важен порядок колонок?
 - 98. Чем B-tree, Hash, GIN и GiST индексы отличаются на практике?
 - 99. Что такое covering index / index-only scan?
 - 100. Что такое partial index и когда он полезен?
 - 101. Что такое expression index?
 - 102. Почему слишком много индексов - проблема?
 - 103. Что такое transaction isolation anomaly на примере lost update?
 - 104. Что такое write skew?
 - 105. Что такое phantom read и почему он важен для бизнес-инвариантов?
 - 106. Чем pessimistic locking отличается от optimistic locking?
 - 107. Что делает SELECT ... FOR UPDATE?
 - 108. Что такое deadlock в БД и как его предотвращать?
 - 109. Почему нельзя делать внешние API-вызовы внутри DB-транзакции?
 - 110. Как правильно использовать defer tx.Rollback() в Go?
 - 111. Как настроить connection pool в Go и какие симптомы неправильной настройки?
 - 112. Что такое prepared statements и parameterized queries?
 - 113. Что такое SQL injection в ORDER BY и как от нее защищаться?
 - 114. Почему SELECT * часто плохая идея?
 - 115. Offset pagination: почему она деградирует?
 - 116. Как проектировать cursor pagination правильно?
 - 117. Как безопасно делать schema migration на большой таблице?
 - 118. Как добавлять NOT NULL колонку с default на большой таблице?
 - 119. Что такое online index creation?
 - 120. Что такое replication lag и чем он опасен?
 - 121. Когда читать с replica, а когда с primary?

122. Что такое read/write splitting и какие у него ловушки?
 123. Что такое sharding и когда он нужен?
 124. Как выбрать shard key?
 125. Чем partitioning отличается от sharding?
 126. Когда полезно partitioning по времени?
 127. Как проектировать soft delete?
 128. Как хранить audit log?
 129. Что такое outbox pattern и почему он связан с БД?
 130. Что такое inbox/idempotent consumer pattern?
 131. Как проектировать уникальность с учетом race conditions?
 132. Как хранить деньги и decimal-значения?
 133. Как моделировать many-to-many связь?
 134. Когда JSON/JSONB в реляционной БД - хорошая идея, а когда плохая?
 135. ORM vs raw SQL: какие trade-off?
 136. Как диагностировать медленный endpoint: БД или приложение?
 137. Что такое lock wait и как его увидеть?
 138. Почему длинные транзакции опасны даже без активной работы?
 139. Что такое MVCC?
 140. Что такое vacuum/bloat в PostgreSQL?
 141. Как проектировать retention и удаление большого объема данных?
 142. Что такое cache-aside и какие риски у кеширования данных из БД?
 143. Что такое cache stampede?
 144. Как выбрать между нормализацией и денормализацией?
 145. Что такое referential integrity и когда нужны foreign keys?
 146. Что такое ON DELETE CASCADE и какие риски?
 147. Как хранить и проверять права доступа в БД-запросах?
 148. Какие вопросы задать кандидату по реальному инциденту с БД?
 149. Практический кейс: endpoint GET /orders стал медленным после роста данных. Как расследовать?
 150. Практический кейс: иногда создание пользователя возвращает 500 вместо "email занят". Что проверить?
 151. Практический кейс: воркеры периодически берут одну и ту же job дважды. Как исправить?
 152. Практический кейс: после релиза выросло число DB connections. Что делать?
 153. Практический кейс: миграция зависла и заблокировала записи. Как действовать?
 154. Какие DB-вопросы стоит считать обязательными для Middle+ Go backend?
- 17.14 Как оценивать ответы

Цель интервью

Оценить, способен ли кандидат уровня Middle+ самостоятельно проектировать и реализовывать production-grade сервисы на Go, принимать инженерные решения, находить компромиссы между простотой и масштабируемостью, разбираться в существующем коде, работать с конкурентностью, базами данных, API, тестированием и эксплуатацией сервисов.

Ожидаемый профиль Middle+

Кандидат уровня Middle+ обычно:

- уверенно пишет идиоматичный Go и понимает стандартную библиотеку;
- умеет проектировать HTTP/gRPC-сервисы, API-контракты и внутренние пакеты;
- понимает goroutines, channels, context, race conditions и базовые паттерны конкурентности;
- работает с SQL/NoSQL, транзакциями, индексами и миграциями;
- пишет unit, integration и table-driven тесты;
- способен диагностировать performance-проблемы и production-инциденты;
- понимает observability: logs, metrics, tracing, alerts;
- умеет обсуждать архитектурные trade-off и защищать технические решения;
- может ревьюить код и быть владельцем части сервиса без постоянного контроля.

Рекомендуемая структура интервью

Общая длительность: 90-120 минут.

Блок	Длительность	Что проверяем
1. Вводная часть и опыт	10 мин	Релевантность опыта, зрелость, зона ответственности
2. Go core и идиоматика	20 мин	Знание языка, стандартной библиотеки, ошибок, интерфейсов
3. Concurrency и context	20 мин	Goroutines, channels, cancellation, races, deadlocks
4. Backend, API и данные	20 мин	HTTP/gRPC, БД, транзакции, надежность
5. Практическая задача	25-35 мин	Код, декомпозиция, тестируемость, edge cases
6. System design light	20 мин	Архитектурное мышление, компромиссы, эксплуатация
7. Поведенческий блок и вопросы кандидата	10 мин	Коммуникация, ownership, работа в команде

1. Вводная часть и опыт

Цели блока

- Понять, какие задачи кандидат реально решал сам.
- Выяснить размер и критичность систем, с которыми он работал.
- Отличить практический production-опыт от поверхностного знакомства с технологиями.

Основные вопросы

1. Расскажите о последнем Go-проекте: какую проблему решал сервис, какая была ваша зона ответственности?

2. Какой самый сложный технический выбор вы делали в этом проекте? Какие альтернативы рассматривали?
3. С какими нагрузками работал сервис: RPS, объем данных, latency, количество пользователей или сообщений?
4. Как была организована разработка: code review, тестирование, CI/CD, релизы, мониторинг?
5. Приведите пример production-инцидента, в расследовании которого вы участвовали. Как нашли причину?
6. Что в Go вам нравится, а что кажется неудобным или ограничивающим?

Сильные сигналы

- Кандидат говорит конкретно: метрики, ограничения, решения, последствия.
- Может объяснить не только что делал, но и почему.
- Осознает стоимость решений: complexity, maintainability, latency, reliability.

Риски

- Описывает опыт общими словами без деталей.
- Не может объяснить архитектуру сервиса, над которым работал.
- Приписывает себе командные решения, но не показывает личный вклад.

2. Go core и идиоматика

Темы для проверки

- packages, visibility, init;
- pointers, values, escape analysis на базовом уровне;
- slices, maps, arrays;
- interfaces и embedding;
- error handling;
- defer, panic, recover;
- generics;
- standard library;
- memory model на практическом уровне;
- heap, stack, escape analysis, allocations и влияние GC.

Вопросы по языку

1. Чем slice отличается от array? Что хранит slice header?
2. Что произойдет при append к slice, если capacity достаточно, и если capacity недостаточно?
3. Почему изменение одного slice может неожиданно изменить другой slice?
4. Можно ли читать из nil map? Можно ли писать в nil map?
5. Что происходит при обращении к отсутствующему ключу map?
6. Почему map в Go не потокобезопасна?
7. Когда стоит использовать pointer receiver, а когда value receiver?
8. Чем interface value с nil concrete value отличается от nil interface?

9. Как Go выбирает, реализует ли тип интерфейс?
10. Для чего нужен empty interface или any? Когда его использование ухудшает дизайн?
11. Как работает defer? В каком порядке выполняются несколько defer?
12. Что будет с аргументами defer: вычисляются сразу или в момент выполнения?
13. Когда допустимо использовать panic/recover в backend-сервисе?
14. Как вы обычно оборачиваете и сравниваете ошибки?
15. Чем errors.Is отличается от errors.As?
16. Когда уместны sentinel errors, а когда лучше typed errors?
17. Что такое zero value и почему это важная идея в Go?
18. Как generics изменили подход к utility-коду? Когда не стоит использовать generics?
19. Какие пакеты стандартной библиотеки вы используете чаще всего?
20. Что такое escape analysis и как она может влиять на производительность?
21. Чем stack отличается от heap в Go?
22. Где хранятся локальные переменные функции и всегда ли локальная переменная находится на stack?
23. Почему возврат указателя на локальную переменную безопасен в Go?
24. Как растёт stack goroutine и почему это важно для большого количества goroutines?
25. Почему утверждение "указатели всегда быстрее" неверно?
26. Как диагностировать лишние heap allocations?

Практические мини-вопросы

Slice aliasing

```
a := []int{1, 2, 3, 4}
b := a[:2]
b = append(b, 99)
fmt.Println(a)
```

Вопросы:

- Что будет выведено?
- Почему?
- Как избежать неожиданного изменения исходного slice?

Ожидаемое направление ответа: кандидат понимает shared backing array и предлагает copy или full slice expression с ограничением capacity.

Nil interface

```
type MyError struct{}

func (e *MyError) Error() string { return "boom" }

func do() error {
    var err *MyError = nil
    return err
}

fmt.Println(do() == nil)
```

Вопросы:

- Что будет выведено?
- Почему error interface не равен nil?

- Как правильно вернуть отсутствие ошибки?

Ожидаемое направление ответа: interface содержит dynamic type и dynamic value; nil должен возвращаться как untyped nil.

3. Concurrency и context

Темы для проверки

- goroutines lifecycle;
- channels: buffered/unbuffered, close, select;
- sync.Mutex, RWMutex, WaitGroup, Once, Cond;
- atomic operations;
- context cancellation and deadlines;
- race conditions;
- worker pools;
- backpressure;
- graceful shutdown.

Основные вопросы

1. Чем goroutine отличается от OS thread?
2. Что может привести к goroutine leak?
3. Как обнаружить goroutine leak в production?
4. Когда использовать channel, а когда mutex?
5. Что произойдет при чтении из закрытого channel?
6. Что произойдет при записи в закрытый channel?
7. Кто должен закрывать channel: отправитель или получатель?
8. Для чего нужен select с default?
9. Как реализовать fan-out/fan-in?
10. Как ограничить количество одновременно выполняющихся задач?
11. Как корректно остановить worker pool?
12. Как работает context.Context и что нельзя хранить в context?
13. Чем context cancellation отличается от timeout/deadline?
14. Как передавать context в функции и методы?
15. Почему нельзя игнорировать ctx.Done() в долгих операциях?
16. Что такое data race? Как race detector помогает, а где его недостаточно?
17. Когда использовать sync/atomic вместо mutex?
18. Что такое backpressure и как его реализовать?
19. Как сделать graceful shutdown HTTP-сервера?
20. Как бы вы защитили сервис от лавинообразного роста goroutines?

Практический concurrency-сценарий

Нужно обработать 10 000 задач, каждая делает HTTP-запрос к внешнему сервису. Внешний сервис нестабилен, есть rate limit, запросы могут зависать.

Попросите кандидата описать решение:

- ограничение параллелизма;
- context timeout на запрос;
- retry policy с backoff;
- обработка частичных ошибок;
- сбор результатов;
- остановка при отмене;
- метрики и логирование;
- защита от goroutine leaks.

Сильный ответ включает worker pool или semaphore, context propagation, bounded queue, timeouts, idempotency considerations, observability и понятную стратегию ошибок.

4. Backend, API и работа с данными

HTTP/gRPC и API

1. Как вы проектируете REST API для нового ресурса?
2. Какие HTTP-коды вы используете для validation error, conflict, unauthorized, forbidden, not found?
3. Как организовать versioning API?
4. Как валидировать входные данные?
5. Где должна жить бизнес-логика: handler, service, repository?
6. Как вы обрабатываете request timeout?
7. Чем middleware отличается от handler wrapper?
8. Как реализовать idempotency для POST-запроса?
9. Когда выбирать gRPC вместо REST?
10. Какие проблемы возникают при backward-incompatible изменениях API?

Базы данных

1. Как устроена транзакция? Что такое commit и rollback?
2. Какие уровни изоляции транзакций вы знаете?
3. Что такое dirty read, non-repeatable read, phantom read?
4. Как избежать lost update?
5. Как работает optimistic locking?
6. Как выбрать индекс для SQL-запроса?
7. Почему индекс может не использоваться?
8. Как анализировать медленный SQL-запрос?
9. Чем INNER JOIN отличается от LEFT JOIN?
10. Что такое N+1 problem и как ее исправить?
11. Как проектировать миграции без downtime?
12. Как хранить и обрабатывать nullable-поля в Go?
13. Когда использовать SQL, а когда NoSQL?
14. Как организовать pagination: offset vs cursor?

15. Как вы проектируете repository layer?

Reliability

1. Что такое timeout, retry и circuit breaker?
2. Почему retries могут ухудшить инцидент?
3. Что такое idempotency key?
4. Как обрабатывать partial failure?
5. Как проектировать взаимодействие с внешним API?
6. Что логировать при ошибке, а что нельзя логировать?
7. Какие метрики нужны для HTTP-сервиса?
8. Как понять, что сервис деградирует до полной поломки?

5. Практическая задача

Выберите одну задачу в зависимости от фокуса вакансии. Для Middle+ важно не только получить работающий код, но и увидеть ход мыслей, декомпозицию, обработку ошибок и тестируемость.

Вариант A: In-memory rate limiter

Задача: реализовать in-memory rate limiter для API по userID.

Требования:

- лимит N запросов за окно времени T;
- потокобезопасность;
- метод `Allow(userID string) bool`;
- очистка старых записей;
- возможность написать unit tests;
- обсуждение поведения при большом количестве пользователей.

Вопросы после реализации:

1. Какой алгоритм вы выбрали: fixed window, sliding window, token bucket? Почему?
2. Какие edge cases есть на границе окна?
3. Как избежать бесконечного роста map?
4. Как сделать реализацию распределенной?
5. Какие метрики вы бы добавили?

Критерии оценки:

- корректная синхронизация;
- простая и понятная структура данных;
- тесты на границы окна;
- отсутствие глобального состояния;
- понимание ограничений in-memory решения.

Вариант B: Concurrent fetch with timeout

Задача: написать функцию, которая параллельно запрашивает данные по списку URL и возвращает успешные ответы.

Пример сигнатуры:

```
func FetchAll(ctx context.Context, client *http.Client, urls []string, limit int) ([]Result, error)
```

Требования:

- ограничить параллелизм параметром `limit`;
- уважать отмену context;
- не допустить goroutine leaks;
- вернуть частичные результаты;
- корректно обработать HTTP errors и timeouts;
- написать table-driven tests с mock server.

Вопросы после реализации:

1. Как остановятся goroutines при отмене context?
2. Что произойдет, если один URL зависнет?
3. Как сохранить порядок результатов?
4. Как добавить retry?
5. Как логировать ошибки без утечки персональных данных?

Вариант C: Transactional service method

Задача: спроектировать сервисный метод перевода денег между счетами.

Требования:

- проверить существование счетов;
- проверить достаточность средств;
- выполнить debit и credit атомарно;
- защититься от concurrent transfers;
- сохранить audit log;
- вернуть доменные ошибки;
- обсудить тестирование.

Вопросы после реализации:

1. Где должна начинаться транзакция?
2. Какой уровень изоляции выбрать?
3. Как избежать deadlock?
4. Как сделать операцию идемпотентной?
5. Как обработать сбой после commit, но до ответа клиенту?

6. System design light

Сценарий

Спроектируйте сервис уведомлений, который принимает события от других сервисов и отправляет email, SMS и push-уведомления.

Что попросить кандидата раскрыть

1. API приема событий.
2. Валидация и нормализация данных.

3. Очередь или брокер сообщений.
4. Хранение статуса уведомлений.
5. Retry и dead-letter queue.
6. Idempotency и дедупликация.
7. Rate limits внешних провайдеров.
8. Шаблоны уведомлений.
9. Observability.
10. Graceful degradation.
11. Horizontal scaling.
12. Безопасность и персональные данные.

Уточняющие вопросы

1. Что произойдет, если провайдер SMS недоступен 30 минут?
2. Как не отправить одно уведомление дважды?
3. Как гарантировать, что событие не потеряется?
4. Какой delivery guarantee выбрать: at-most-once, at-least-once или exactly-once?
5. Какие таблицы или коллекции нужны?
6. Какие индексы понадобятся?
7. Как мониторить задержку доставки?
8. Как разделить критичные уведомления от маркетинговых?
9. Как безопасно хранить шаблоны и пользовательские данные?
10. Как тестировать такую систему?

Сильный ответ

Кандидат не пытается сразу построить чрезмерно сложную систему, а уточняет требования, выделяет критичные потоки, объясняет trade-off между простотой, надежностью и стоимостью. В ответе должны появиться idempotency, retries, DLQ, outbox или аналогичный паттерн для надежной публикации событий, метрики, structured logs и понятная модель данных.

7. Тестирование и качество

Вопросы

1. Как вы обычно пишете unit tests в Go?
2. Что такое table-driven tests и почему они популярны в Go?
3. Как тестировать код, который зависит от времени?
4. Как тестировать HTTP handler?
5. Как тестировать интеграцию с базой данных?
6. Когда использовать mocks, fakes и testcontainers?
7. Как проверять race conditions?
8. Что должно быть в CI pipeline для Go-сервиса?
9. Какой уровень покрытия тестами вы считаете полезным?
10. Как вы относитесь к snapshot tests?

Практическое обсуждение

Дайте кандидату небольшой фрагмент плохо тестируемого кода: прямой вызов `time.Now`, глобальный HTTP client, работа с БД внутри handler. Попросите объяснить, как сделать код тестируемым.

Ожидаемые идеи:

- dependency injection;
- интерфейсы на границах;
- clock abstraction;
- httptest;
- transaction rollback в интеграционных тестах;
- разделение handler/service/repository.

8. Performance и profiling

Вопросы

1. Как понять, что Go-сервис упирается в CPU, память, БД или сеть?
2. Как использовать pprof?
3. Какие профили вы знаете: cpu, heap, goroutine, mutex, block?
4. Что такое allocation и почему лишние allocations важны?
5. Когда стоит оптимизировать код?
6. Как benchmark отличается от unit test?
7. Как написать корректный benchmark в Go?
8. Что может привести к высокому GC pressure?
9. Как искать memory leak в Go?
10. Какие типичные причины latency spikes?

Сильные сигналы

- Кандидат начинает с измерений, а не с догадок.
- Умеет пользоваться pprof, metrics и tracing.
- Понимает, что преждевременная оптимизация может ухудшить дизайн.

9. Observability и эксплуатация

Вопросы

1. Какие structured logs нужны в HTTP-сервисе?
2. Что должно быть в request ID или trace ID?
3. Какие RED/USE-метрики вы знаете?
4. Какие alerts вы бы настроили для backend-сервиса?
5. Чем symptoms-based alerts лучше cause-based alerts?
6. Как логировать ошибки внешних API?
7. Как связаны tracing и logs?

8. Что должно быть в health check?
9. Чем readiness отличается от liveness?
10. Как организовать graceful shutdown в Kubernetes?

10. Security

Вопросы

1. Как хранить пароли пользователей?
2. Почему нельзя логировать access tokens и персональные данные?
3. Чем authentication отличается от authorization?
4. Как устроен JWT? Какие у него риски?
5. Что такое CSRF и когда он актуален?
6. Что такое SQL injection и как от него защищаться?
7. Как валидировать входные данные?
8. Как хранить secrets для сервиса?
9. Как ограничить blast radius при компрометации сервиса?
10. Какие security checks можно добавить в CI?

11. Code review

Формат

Покажите кандидату pull request или фрагмент кода с намеренными проблемами:

- business logic в HTTP handler;
- отсутствие context propagation;
- бесконечные retries;
- игнорирование ошибок;
- race condition;
- нет timeout у HTTP client;
- SQL-запрос без индекса;
- логирование чувствительных данных;
- непроверенные edge cases.

Вопросы

1. Какие проблемы вы видите?
2. Какие замечания обязательны к исправлению, а какие можно оставить как улучшение?
3. Как бы вы сформулировали комментарий в code review?
4. Какие тесты нужно добавить?
5. Что можно упростить без изменения поведения?

Сильный ответ

Кандидат приоритизирует риски, не заикливается на стиле, объясняет влияние проблемы на пользователей или эксплуатацию, предлагает конкретные исправления и умеет коммуницировать без агрессии.

12. Поведенческий блок

Вопросы

1. Расскажите о случае, когда вы не согласились с техническим решением команды. Что сделали?
2. Когда вы в последний раз сильно упростили решение? Что помогло увидеть простоту?
3. Приведите пример, когда вы ошиблись в production. Как исправляли последствия?
4. Как вы принимаете решение между быстрым fix и правильным refactoring?
5. Как вы помогали менее опытному разработчику?
6. Что для вас хороший code review?
7. Как вы действуете, если задача плохо сформулирована?
8. Как вы сообщаете менеджеру или команде о техническом риске?
9. Что вы делаете, если не успеваете закончить задачу в ожидаемый срок?
10. Как вы поддерживаете качество в условиях давления на сроки?

13. Матрица оценки

Оценивайте каждый блок по шкале 1-4.

Балл	Интерпретация
1	Знания фрагментарные, много критичных пробелов, нужен постоянный контроль
2	Уровень ближе к Middle-, решает типовые задачи, но слабо видит риски
3	Уверенный Middle/Middle+, самостоятельно решает production-задачи
4	Сильный Middle+/Senior-, системно мыслит, предвидит проблемы, усиливает команду

Компетенции

Компетенция	Вес	Что считать хорошим результатом
Go core и идиоматика	20%	Понимает язык, ошибки, interfaces, slices/maps, стандартную библиотеку
Concurrency	20%	Уверенно работает с goroutines, channels, context, races
Backend и данные	20%	Проектирует API, транзакции, надежную работу с внешними системами
Практическое кодирование	20%	Пишет простой, корректный, тестируемый код
Архитектура и эксплуатация	15%	Видит trade-off, observability, reliability, scaling
Коммуникация и ownership	5%	Ясно объясняет решения, принимает ответственность

Decision guide

- Strong hire: в большинстве блоков 3-4, нет критичных пробелов в concurrency, данных и production-надежности.
- Hire: стабильные 3, отдельные слабые зоны компенсируются опытом и обучаемостью.
- Lean no hire: много ответов на уровне теории, слабая практика с context, транзакциями или тестами.
- No hire: не может объяснить базовые конструкции Go, не видит race conditions, игнорирует ошибки и эксплуатационные риски.

14. Красные флаги

- Не понимает nil interface и error handling в Go.
- Не умеет объяснить lifecycle goroutine и риски goroutine leaks.
- Предлагает unbounded goroutines для внешних запросов.
- Игнорирует context cancellation.
- Использует panic для обычных бизнес-ошибок.
- Не пишет тесты или считает их необязательными.
- Не умеет объяснить транзакции и конкурентное обновление данных.
- Считает retries универсальным решением без timeout и backoff.
- Не различает authentication и authorization.
- Не может рассказать о собственных ошибках и выводах.

15. Вопросы кандидата к интервьюеру

Оставьте 5-10 минут на вопросы кандидата. Хорошие вопросы кандидата тоже являются сигналом зрелости.

На что обратить внимание:

- спрашивает о домене, архитектуре, нагрузке, команде;
- интересуется качеством процессов, on-call, observability;
- уточняет ожидания от роли и зоны ответственности;
- задает вопросы о техническом долге и планах развития.

16. Итоговый шаблон заметок интервьюера

Кандидат:
Дата:
Интервьюеры:

Краткое резюме:

Go core:
Оценка 1-4:
Заметки:

Concurrency:
Оценка 1-4:
Заметки:

Backend/API/DB:
Оценка 1-4:
Заметки:

Практическая задача:

Оценка 1-4:

Заметки:

System design:

Оценка 1-4:

Заметки:

Коммуникация и ownership:

Оценка 1-4:

Заметки:

Сильные стороны:

Риски:

Рекомендация:

Strong hire / Hire / Lean no hire / No hire

Аргументация:

17. Подробные вопросы и ответы для интервьюера

Этот раздел можно использовать как ключ к интервью. Ответы не требуют дословного совпадения: сильный кандидат объясняет идею своими словами, приводит примеры из production и видит ограничения решения.

17.1 Go core и идиоматика

1. Чем slice отличается от array? Что хранит slice header?

Ожидаемый ответ: array в Go - это значение фиксированной длины, и длина входит в тип: `[3]int` и `[4]int` являются разными типами. Array при присваивании или передаче в функцию копируется целиком, если не передавать указатель. Slice - это дескриптор над backing array. Он не хранит элементы сам, а содержит три части: указатель на первый доступный элемент массива, длину `len` и емкость `cap`.

Подробное объяснение: когда мы передаем slice в функцию, копируется только slice header. Поэтому функция получает собственную копию `len/cap/pointer`, но этот `pointer` указывает на тот же backing array. Изменение элемента `s[i]` видно вызывающему коду, потому что меняется общий массив. Но изменение самой длины slice, например `s = s[:0]` или `s = append(s, x)`, меняет локальную копию header и не обязательно видно снаружи.

Пример для обсуждения:

```
func change(s []int) {
    s[0] = 10
    s = append(s, 20)
}

func main() {
    xs := []int{1, 2, 3}
    change(xs)
    fmt.Println(xs)
}
```

Кандидат должен объяснить, что первый элемент изменится, а новая длина `xs` снаружи не появится, потому что header был скопирован. При этом `append` мог изменить backing array, если `capacity` хватило.

Что важно услышать: slice - маленькая структура, но разделяемый backing array создает aliasing. Для безопасной передачи ownership иногда нужен `copy`, `slices.Clone` или ограничение `capacity` через full slice expression.

Уточняющие вопросы:

- Почему `[]T` часто передают по значению, а не `*[]T`?

- В каких случаях `*[N]T` может быть полезнее `slice`?
- Что произойдет с памятью, если маленький `slice` ссылается на огромный `backing array`?

2. Что происходит при `append` к `slice` при достаточной и недостаточной `capacity`?

Ожидаемый ответ: если `cap` достаточно, `append` записывает новые элементы в существующий `backing array` и возвращает новый `slice header` с увеличенным `len`. Если `cap` недостаточно, `runtime` выделяет новый `backing array`, копирует старые элементы, добавляет новые и возвращает `slice header`, указывающий уже на новый массив.

Подробное объяснение: `append` никогда не обязан изменять исходный `header` на месте, поэтому результат всегда нужно присваивать: `s = append(s, x)`. Старые `slice`, которые ссылались на прежний `backing array`, не начинают автоматически ссылаться на новый. Это часто создает баги, когда код делает `append` в `helper-функции`, но не возвращает новый `slice`.

Пример ловушки:

```
func addBad(s []int) {
    s = append(s, 4)
}

func addGood(s []int) []int {
    return append(s, 4)
}
```

`addBad` может даже записать 4 в общий `backing array`, но вызывающий код не увидит новую длину. `addGood` явно возвращает обновленный `header`.

Что важно услышать: кандидат не должен полагаться на конкретный алгоритм роста `capacity` как на контракт. Важно понимать поведенческий контракт: старый `backing array` может быть переиспользован или заменен.

Красные флаги:

- "append всегда создает новый массив";
- "append всегда меняет исходный `slice`";
- кандидат забывает присваивать результат `append`.

3. Почему изменение одного `slice` может неожиданно изменить другой `slice`?

Ожидаемый ответ: несколько `slice` могут ссылаться на один `backing array`. Если один `slice` меняет элемент или делает `append` в пределах своей `capacity`, он может перезаписать данные, которые видны через другой `slice`.

Подробное объяснение: `slicing` не копирует данные. Выражение `b := a[:2]` создает новый `header`, но `backing array` остается тем же. Если `b` имеет `capacity` больше длины, `append` может писать за пределами `len(b)`, но внутри `cap(b)`, то есть в элементы, которые с точки зрения другого `slice` уже являются валидными.

Пример:

```
base := []int{1, 2, 3, 4}
left := base[:2]
right := base[2:]
left = append(left, 99)
fmt.Println(base) // [1 2 99 4]
fmt.Println(right) // [99 4]
```

Как избежать: если нужно изолировать данные, использовать `copy` или `slices.Clone`. Если нужно разрешить `append`, но запретить запись в хвост исходного массива, можно ограничить `capacity`: `left := base[:2:2]`. Тогда следующий `append` вынудит `runtime` выделить новый `backing array`.

Уточняющие вопросы:

- Почему `copy(dst, src)` возвращает число скопированных элементов?

- Как удалить элемент из slice без утечки ссылок на большие объекты?
- Почему `s = s[:0]` может удерживать память?

4. Можно ли читать из nil map? Можно ли писать в nil map?

Ожидаемый ответ: читать из nil map можно: вернется zero value типа значения. В форме `value, ok := m[key]` флаг `ok` будет `false`. Писать в nil map нельзя: будет `panic assignment to entry in nil map`.

Подробное объяснение: nil map ведет себя как пустая map для чтения, но у нее нет внутренней структуры `buckets` для записи. Поэтому перед записью map нужно инициализировать через `make(map[K]V)` или литерал `map[K]V{}`.

Пример:

```
var counters map[string]int
fmt.Println(counters["x"]) // 0
counters["x"]++           // panic
```

Что важно услышать: nil slice и nil map ведут себя по-разному. В nil slice можно делать `append`, потому что `append` вернет новый slice с выделенным `backing array`. В nil map запись невозможна без `make`.

Практический сигнал: кандидат понимает, почему поля типа `map` в `struct` часто инициализируют в `constructor`-е, если ожидается запись.

5. Что происходит при обращении к отсутствующему ключу map?

Ожидаемый ответ: возвращается zero value для типа значения. Чтобы отличить отсутствующий ключ от реально сохраненного zero value, используют `comma-ok` форму: `value, ok := m[key]`.

Подробное объяснение: для `map[string]int` отсутствующий ключ возвращает `0`, но `0` может быть валидным значением. Для `map[string]*User` отсутствующий ключ возвращает `nil`, но `nil` тоже может быть намеренно сохраненным значением, если тип значения `pointer`. Поэтому `ok` важен, когда семантика отсутствия отличается от zero value.

Пример:

```
attempts := map[string]int{"alice": 0}
value, ok := attempts["alice"]
fmt.Println(value, ok) // 0 true
value, ok = attempts["bob"]
fmt.Println(value, ok) // 0 false
```

Уточняющий вопрос: как бы кандидат спроектировал `map-set` в Go? Ожидаемый вариант: `map[string]struct{}` или `map[string]bool`, с объяснением `trade-off` читаемости и памяти.

6. Почему map в Go не потокобезопасна?

Ожидаемый ответ: стандартная map не синхронизирует доступ к своей внутренней структуре. Одновременная запись из нескольких `goroutine` или чтение одновременно с записью приводят к `data race` и могут закончиться `panic concurrent map read and map write`.

Подробное объяснение: map внутри может перераспределять `buckets`, менять `metadata`, перемещать элементы при росте. Если другая `goroutine` одновременно читает или пишет, она может увидеть неконсистентное состояние. Даже если `panic` не произошло, программа содержит `race` и ее поведение некорректно.

Как исправлять:

- `sync.Mutex` для простого shared state;
- `sync.RWMutex`, если много чтений и мало записей, но нужно учитывать `contention` и `writer starvation`;
- `ownership` через одну `goroutine` и `channels`, если модель естественно `event-driven`;
- `sync.Map` для специальных сценариев: `read-mostly cache`, независимые ключи, редкие `updates`.

Сильный ответ: кандидат не предлагает `sync.Map` как универсальную замену. Он уточняет `workload`, инварианты и требования к атомарности операций. Например, "прочитать значение, изменить поле и записать обратно" может требовать `mutex` даже при использовании `concurrent container`.

7. Когда стоит использовать `pointer receiver`, а когда `value receiver`?

Ожидаемый ответ: `pointer receiver` нужен, если метод изменяет состояние объекта, если структура большая и копирование дорого, если тип содержит `sync primitives`, если нужно сохранить `identity`, или если часть методов уже использует `pointer receiver` и важно единообразие `method set`. `Value receiver` подходит для маленьких `immutable-like` типов, где копирование дешево и ожидаемо.

Подробное объяснение: `receiver` передается как обычный аргумент функции. `Value receiver` копирует значение. Если `struct` содержит `sync.Mutex`, копирование почти всегда ошибка, потому что копируется состояние `lock-a`. Если `struct` маленькая, например `type Point struct { X, Y int }`, `value receiver` может быть проще и безопаснее.

`Method set`: тип `T` имеет методы с `value receiver`. Тип `*T` имеет методы и с `value receiver`, и с `pointer receiver`. Поэтому если интерфейс требует метод с `pointer receiver`, значение `T` может не реализовывать интерфейс, а `*T` реализует.

Пример:

```
type Counter struct{ n int }
func (c *Counter) Inc() { c.n++ }
func (c Counter) Value() int { return c.n }
```

`Inc` должен быть `pointer receiver`, иначе изменится копия. `Value` может быть `value receiver`, если структура маленькая и не содержит `lock`.

Уточняющие вопросы:

- Почему не стоит смешивать `receiver types` без причины?
- Что произойдет, если `value receiver` используется у `struct` с `map` или `slice` полями?

8. Чем `interface value с nil concrete value` отличается от `nil interface`?

Ожидаемый ответ: `interface value` в Go содержит `dynamic type` и `dynamic value`. `Interface` равен `nil` только когда оба компонента `nil`. Если в `interface` положить `typed nil`, например `(*MyError)(nil)`, `dynamic type` будет `*MyError`, а `dynamic value` будет `nil`. Такой `interface` уже не равен `nil`.

Подробное объяснение: это особенно часто проявляется с `error`. Функция может вернуть `typed nil` как `error`, и вызывающий код увидит `err != nil`, хотя внутри `pointer nil`. Поэтому отсутствие ошибки нужно возвращать как обычный `untyped nil`.

Пример:

```
type MyError struct{}
func (*MyError) Error() string { return "boom" }

func do() error {
    var err *MyError = nil
    return err
}

fmt.Println(do() == nil) // false
```

Как исправлять:

```
func do() error {
    var err *MyError = nil
    if err == nil {
        return nil
    }
    return err
}
```

Сильный ответ: кандидат может объяснить не только "так устроен interface", но и practical impact: неверная обработка ошибок, неожиданные 500, ложные retries, падения при вызове метода на nil receiver.

9. Как Go выбирает, реализует ли тип интерфейс?

Ожидаемый ответ: реализация интерфейса в Go неявная. Тип реализует интерфейс, если его method set содержит все методы интерфейса с точно совпадающими именами, параметрами и возвращаемыми типами. Никакого implements писать не нужно.

Подробное объяснение: интерфейс описывает поведение, а не наследование. Это позволяет объявлять маленькие интерфейсы на стороне потребителя. Например, функция, которой нужно только читать bytes, может принимать io.Reader, не зная конкретный тип.

Method set пример:

```
type Flusher interface { Flush() error }
type Buffer struct{}
func (b *Buffer) Flush() error { return nil }
```

*Buffer реализует Flusher, а Buffer как значение - нет, потому что Flush объявлен на pointer receiver.

Хорошая практика: интерфейсы обычно лучше держать маленькими. Большой интерфейс затрудняет тесты и заставляет типы реализовывать лишнее. Частый Go-подход: "accept interfaces, return concrete types".

Красный флаг: кандидат описывает интерфейсы как классическое наследование или предлагает заранее создавать большие интерфейсы для всех сервисов.

10. Для чего нужен empty interface или any? Когда его использование ухудшает дизайн?

Ожидаемый ответ: any - это alias для interface{}. Он означает "значение любого типа". Он полезен на границах с динамическими данными: JSON, логирование, generic containers, compatibility layers, metadata. Но чрезмерное использование any убирает compile-time guarantees и переносит ошибки в runtime.

Подробное объяснение: если функция принимает any, ее контракт становится менее очевидным. Внутри почти всегда появляется type assertion, type switch или reflection. Это может быть оправдано в инфраструктурном коде, но в бизнес-логике часто означает слабую модель типов.

Пример плохого API:

```
func Create(entity any) error
```

Такой контракт не говорит, какие entity допустимы. Лучше иметь конкретный тип или маленький интерфейс поведения.

Пример оправданного использования:

```
logger.Info("request failed", "status", statusCode, "error", err)
```

Structured logger принимает пары ключ-значение разных типов, и any там естественен.

Уточняющие вопросы:

- Чем any отличается от type parameters в generics?
- Когда type assertion может panic и как использовать comma-ok форму?

11. Как работает defer? В каком порядке выполняются несколько defer?

Ожидаемый ответ: defer откладывает вызов функции до выхода из текущей функции. Если defer-ов несколько, они выполняются в обратном порядке - LIFO. Аргументы deferred-вызова вычисляются сразу в момент выполнения строки с defer, а не в момент фактического вызова.

Подробное объяснение: defer удобен для освобождения ресурсов рядом с их захватом: mu.Lock(); defer mu.Unlock(), file.Close(), rows.Close(), cancel(), rollback транзакции. Это снижает риск

забыть `cleanup` на раннем `return`.

Пример с аргументами:

```
x := 1
defer fmt.Println(x)
x = 2
// напечатает 1
```

Если deferred function замыкает переменную без передачи аргументом, она увидит актуальное значение на момент выполнения:

```
x := 1
defer func() { fmt.Println(x) }()
x = 2
// напечатает 2
```

Практические нюансы: `defer` внутри длинного цикла откладывает `cleanup` до выхода из всей функции. Если в цикле открываются файлы или `rows`, лучше вынести тело в отдельную функцию или закрывать явно в конце итерации.

12. Что будет с аргументами `defer`: вычисляются сразу или в момент выполнения?

Ожидаемый ответ: `receiver` и аргументы deferred call вычисляются сразу. Сам вызов выполняется позже. Это относится и к методам: `receiver` тоже фиксируется в момент `defer`.

Подробное объяснение: это важно для логирования, метрик и ошибок. Если написать `defer observe(time.Now())`, то в `defer` попадет уже вычисленное время старта? В таком виде нужно внимательно смотреть, что именно вычисляется сразу. Частый паттерн для `latency`:

```
start := time.Now()
defer func() {
    metrics.Observe(time.Since(start))
}()
```

Здесь `start` вычислен сразу, а `time.Since(start)` - при выходе из функции.

Пример ошибки:

```
for _, f := range files {
    file, _ := os.Open(f)
    defer file.Close()
}
```

Все файлы закроются только при выходе из функции, а не в конце каждой итерации. Для большого числа файлов это может исчерпать `descriptors`.

13. Когда допустимо использовать `panic/recover` в backend-сервисе?

Ожидаемый ответ: `panic` не должен быть механизмом обычного control flow и обработки бизнес-ошибок. Его допустимо использовать для programmer errors, нарушенных инвариантов, невозможной инициализации, или в местах, где продолжать работу опасно. `recover` обычно ставят на границе goroutine/request: HTTP middleware, worker wrapper, job runner.

Подробное объяснение: обычные ошибки вроде "пользователь не найден", "недостаточно средств", "невалидный input" должны возвращаться как `error` и мапиться в доменные/HTTP ошибки. `Panic` для таких ситуаций ухудшает читаемость, ломает транзакции, усложняет тесты и может уронить процесс, если `recovery` нет.

Важная деталь: `recover` работает только внутри deferred function в той же goroutine, где произошел `panic`. Recovery middleware HTTP-сервера не поймает `panic` в goroutine, которую handler запустил отдельно.

Хороший production-подход: recovery middleware логирует stack trace, request id, route, user/tenant id при допустимости, возвращает 500 без внутренних деталей и увеличивает panic metric. После `panic` в рамках запроса нельзя пытаться "как будто все нормально" продолжить изменять shared state.

14. Как вы обычно оборачиваете и сравниваете ошибки?

Ожидаемый ответ: при передаче ошибки выше по стеку добавляют контекст через `fmt.Errorf("operation: %w", err)`. Для проверки причины используют `errors.Is`, для извлечения `typed error` - `errors.As`. Для новых доменных ошибок используют `sentinel errors` или `custom types`, если нужны поля.

Подробное объяснение: ошибка должна отвечать на два вопроса: что можно показать/решить на текущем уровне и какая исходная причина нужна для диагностики. `Repository` может вернуть `sql.ErrNoRows`, `service` преобразует ее в доменную `ErrUserNotFound`, `handler` мапит в 404. При этом внутренние SQL details не должны уходить клиенту.

Пример:

```
var ErrUserNotFound = errors.New("user not found")

func (r *Repo) Get(ctx context.Context, id string) (*User, error) {
    user, err := r.query(ctx, id)
    if errors.Is(err, sql.ErrNoRows) {
        return nil, ErrUserNotFound
    }
    if err != nil {
        return nil, fmt.Errorf("query user %s: %w", id, err)
    }
    return user, nil
}
```

Что важно услышать: кандидат не сравнивает строки ошибок, не теряет исходную ошибку через `%v`, не логирует одну и ту же ошибку на каждом слое без добавления смысла.

15. Чем `errors.Is` отличается от `errors.As`?

Ожидаемый ответ: `errors.Is` проверяет, содержит ли `error chain` конкретную ошибку или ошибку, которая считает себя эквивалентной через метод `Is`. `errors.As` ищет в `chain` ошибку определенного типа и, если находит, записывает ее в переданную переменную.

Подробное объяснение: `errors.Is` используют для категорий и `sentinel errors`: `not found`, `conflict`, `canceled`. `errors.As` используют, когда нужны структурированные поля `typed error`: `validation fields`, `retryable flag`, `external status code`.

Пример:

```
if errors.Is(err, context.DeadlineExceeded) {
    // timeout
}

var validationErr *ValidationError
if errors.As(err, &validationErr) {
    // можно прочитать validationErr.Fields
}
```

Типичная ошибка: передавать в `errors.As` не `pointer to target` или пытаться использовать `errors.Is` там, где нужен доступ к полям `typed error`.

16. Когда уместны `sentinel errors`, а когда лучше `typed errors`?

Ожидаемый ответ: `sentinel errors` подходят для небольшого набора стабильных категорий: `ErrNotFound`, `ErrConflict`, `ErrUnauthorized`. `Typed errors` лучше, когда ошибка несет данные: список невалидных полей, внешний `status code`, `retry-after`, `operation name`, `resource id`.

Подробное объяснение: `sentinel error` становится частью API пакета. Если внешний код делает `errors.Is(err, ErrNotFound)`, вы обязаны сохранять этот контракт. `Typed error` тоже является контрактом, если экспортируется, но он позволяет расширять информацию без парсинга строки.

Пример `typed error`:

```
type ValidationError struct {
    Fields map[string]string
}

func (e *ValidationError) Error() string { return "validation failed" }
```

Хороший дизайн: внутри сервиса можно иметь богатые typed errors, а на границе API преобразовывать их в стабильный error response. Не стоит раскрывать клиенту internal SQL/network messages.

17. Что такое zero value и почему это важная идея в Go?

Ожидаемый ответ: zero value - значение по умолчанию для типа: 0, false, "", nil, struct с zero-полями. Go поощряет типы, которые полезны или хотя бы безопасны в zero value. Например, `sync.Mutex` и `bytes.Buffer` готовы к использованию без constructor-a.

Подробное объяснение: zero value снижает количество обязательной инициализации и делает API проще. Но не каждый тип обязан быть полностью функциональным в zero value. Если у типа есть обязательные зависимости, constructor может быть нужен для соблюдения инвариантов.

Пример хорошего zero value:

```
var b bytes.Buffer
b.WriteString("hello")
```

Пример осторожности: struct с map-полем в zero value имеет nil map. Читать можно, писать нельзя. Если тип обещает метод Add, constructor должен инициализировать map или метод должен лениво делать make.

Уточняющий вопрос: как спроектировать тип так, чтобы его zero value был безопасен, но ошибки конфигурации не скрывались?

18. Как generics изменили подход к utility-коду? Когда не стоит использовать generics?

Ожидаемый ответ: generics позволяют писать типобезопасные функции и структуры для повторяющихся алгоритмов: set, stack, optional-like containers, helpers для slices/maps, numeric algorithms. До generics для этого часто использовали `interface{}`, code generation или дублирование.

Подробное объяснение: generics хороши, когда логика действительно одинакова для разных типов, а type parameter сохраняет static type safety. Но generics не должны заменять интерфейсы поведения. Если функция работает с "тем, что умеет читать", лучше `io.Reader`, а не type parameter. Если типов всего два и generic abstraction делает код сложнее, дублирование может быть дешевле.

Пример уместного generic-кода:

```
func Contains[T comparable](xs []T, target T) bool {
    for _, x := range xs {
        if x == target {
            return true
        }
    }
    return false
}
```

Пример сомнительного подхода: делать generic service/repository для всей бизнес-логики, если из-за этого исчезают явные доменные операции и ошибки.

Что важно услышать: кандидат понимает constraints (`comparable`, union constraints, ~ для underlying type), но не пытается использовать generics как наследование или замену полиморфизма через интерфейсы.

19. Какие пакеты стандартной библиотеки вы используете чаще всего?

Ожидаемый ответ: для backend-разработчика ожидаемо услышать `context`, `net/http`, `encoding/json`, `database/sql`, `errors`, `fmt`, `time`, `sync`, `sync/atomic`, `io`, `os`, `bytes`, `strings`, `strconv`, `testing`, `httptest`, `log/slog`, `crypto/*` по необходимости.

Подробное объяснение: цель вопроса - проверить не список названий, а глубину практического использования standard library. Хороший кандидат может рассказать, как делает HTTP request с

context, как тестирует handler через `httptest`, как использует `errors.Is/As`, почему закрывает `resp.Body`, как настраивает `http.Client` timeouts.

Уточняющие вопросы:

- Чем `http.Client.Timeout` отличается от context timeout?
- Почему важно вычитывать/закрывать `resp.Body`?
- Когда использовать `json.Decoder`, а когда `json.Unmarshal`?
- Что появление `log/slog` изменило в structured logging?

Сильный ответ: кандидат опирается на standard library до подключения внешних зависимостей и понимает ее ограничения.

20. Что такое escape analysis и как она может влиять на производительность?

Ожидаемый ответ: escape analysis - анализ компилятора, который решает, может ли значение жить на stack или должно быть размещено в heap. Если значение может использоваться после выхода из функции или compiler не может доказать безопасность stack allocation, оно escape-ится в heap.

Подробное объяснение: stack allocation дешевле и освобождается вместе с stack frame. Heap allocation дороже и увеличивает работу GC. Но ручная оптимизация ради stack allocation без измерений часто ухудшает читаемость и не дает выигрыша.

Примеры причин escape:

- возврат указателя на локальную переменную;
- сохранение значения в interface, если compiler не может оптимизировать;
- захват переменной closure, живущей дольше функции;
- слишком большой объект или неочевидный lifetime.

Как смотреть: `go build -gcflags=-m` показывает решения компилятора. Для реального impact нужны benchmarks (`go test -bench -benchmem`) и heap/cpu profiles.

Сильный ответ: кандидат говорит: сначала измерить allocations и latency, потом оптимизировать горячие paths. Он не делает вывод "указатели всегда быстрее". Иногда value semantics быстрее и проще, потому что меньше heap allocations и лучше cache locality.

21. Чем stack отличается от heap в Go?

Ожидаемый ответ: stack используется для данных с понятным временем жизни внутри выполнения функции: локальные значения, параметры, return slots и служебные данные вызовов. Heap используется для данных, которые должны жить дольше текущего stack frame или чей lifetime компилятор не может безопасно ограничить stack-ом. Stack дешевле: выделение обычно сводится к сдвигу stack pointer, а освобождение происходит сразу при возврате из функции. Heap дороже: объект должен быть учтен allocator-ом и позже обработан garbage collector-ом.

Подробное объяснение: в Go разработчик не выбирает напрямую `stack` или `heap`, как в языках с ручным управлением памятью. Решение принимает компилятор через escape analysis. Поэтому корректнее говорить не "я создал переменную на stack", а "компилятор может разместить это значение на stack, если докажет, что оно не escape-ится".

Пример для обсуждения:

```
func local() int {
    x := 10
    return x
}

func escaped() *int {
    x := 10
    return &x
}
```

В первом случае `x` обычно может жить на `stack`. Во втором `x` должен продолжить жить после возврата функции, поэтому компилятор разместит его так, чтобы указатель оставался валидным, чаще всего на `heap`.

Что важно услышать: `stack/heap` - это не часть семантики Go-программы, а деталь реализации, влияющая на производительность. Семантически оба варианта безопасны; разница в стоимости `allocations` и `GC pressure`.

22. Где хранятся локальные переменные функции и всегда ли локальная переменная находится на `stack`?

Ожидаемый ответ: локальная переменная не обязана находиться на `stack`. Если она не `escape`-ится и ее `lifetime` понятен компилятору, она может быть размещена на `stack` или даже в `register`-е/оптимизирована. Если переменная `escape`-ится, она будет размещена в `heap` или иначе сохранена так, чтобы оставаться доступной.

Подробное объяснение: слово "локальная" описывает область видимости в исходном коде, а не место хранения в памяти. Переменная может быть видима только внутри функции, но при этом попасть в `heap`, если ссылка на нее уходит наружу через `return`, `closure`, `interface`, `goroutine` или структуру с более долгим временем жизни.

Примеры причин `escape`:

- функция возвращает `&localVar`;
- `closure` захватывает переменную и живет дольше функции;
- значение передается в `interface`, и компилятор не может доказать конкретный `lifetime`;
- `pointer` сохраняется в структуру, `cache`, `map` или `global variable`;
- переменная используется `goroutine`, которая может выполняться после возврата функции.

Сильный ответ: кандидат отделяет `scope` от `lifetime` и понимает, что решение принимает компилятор. Он не делает выводы только по синтаксису `&x` или `new(T)` без проверки `escape analysis`.

23. Почему возврат указателя на локальную переменную безопасен в Go?

Ожидаемый ответ: это безопасно, потому что Go compiler видит, что адрес локальной переменной возвращается наружу, и перемещает значение в `heap` или обеспечивает его корректное время жизни. В Go не возникает `dangling pointer` на `stack frame`, как в языках, где можно вернуть адрес локальной переменной вручную без защиты.

Подробное объяснение: безопасность не означает бесплатность. Такой код корректен, но может добавить `heap allocation`. Если это происходит в горячем `path`, `allocations` могут увеличить нагрузку на `GC` и `latency`. Поэтому вопрос не "можно ли так писать", а "есть ли это на `performance-critical` пути и измерили ли мы стоимость".

Пример:

```
func NewUserID(value string) *string {
    id := strings.TrimSpace(value)
    return &id
}
```

Код безопасен: `id` не исчезнет после возврата. Но если такая функция вызывается миллионы раз в секунду, стоит проверить `allocations` и, возможно, изменить API на возврат значения.

Красный флаг: кандидат говорит, что такой код вернет указатель на уничтоженную `stack`-память. Это неверно для Go и показывает перенос ментальной модели из C/C++ без учета Go compiler/runtime.

24. Как растёт `stack goroutine` и почему это важно для большого количества `goroutines`?

Ожидаемый ответ: goroutine начинается с небольшого stack, который runtime может увеличивать и, при необходимости, перемещать. Это одна из причин, почему goroutines дешевле OS threads: не нужно заранее резервировать большой stack для каждой goroutine. Но stack не бесплатен: глубокая рекурсия, большие локальные значения и огромное число goroutines увеличивают memory footprint.

Подробное объяснение: Go runtime проверяет, хватает ли stack для вызова функции, и может вырастить stack. При росте stack runtime копирует данные в новый участок памяти и обновляет ссылки, которые он умеет отслеживать. Поэтому Go накладывает ограничения на unsafe-паттерны и требует, чтобы runtime мог корректно понимать pointers.

Практические выводы:

- goroutine дешевле thread, но unbounded goroutines все равно опасны;
- большие локальные массивы или структуры могут раздувать stack frame;
- глубокая рекурсия в Go может быть проблемой, потому что tail-call optimization не является тем, на что стоит рассчитывать;
- memory profile и goroutine profile помогают понять, где растет потребление памяти.

Уточняющий вопрос: если сервис создает goroutine на каждый входящий job без лимита, проблема может быть не только в CPU, но и в stack memory, scheduler overhead, внешних connections и backpressure.

25. Почему утверждение "указатели всегда быстрее" неверно?

Ожидаемый ответ: указатель может избежать копирования большого значения, но он также может привести к heap allocation, дополнительной indirection, худшей cache locality и большей нагрузке на GC. Для маленьких структур value semantics часто быстрее, проще и безопаснее.

Подробное объяснение: производительность зависит от размера типа, frequency вызовов, escape analysis, cache locality, mutability и API-контракта. Передача int, time.Time или небольшой struct по значению часто нормальна. Передача большой структуры или структуры с mutex/copy-sensitive полями требует осторожности.

Пример обсуждения:

```
type Point struct { X, Y int }
func Distance(a, b Point) float64 { ... }
```

Здесь pointer receiver/parameters вряд ли помогут. А для большой структуры с большим массивом внутри копирование может быть дорогим. Но если pointer заставит объект escape-иться в heap, итог может стать хуже.

Сильный ответ: кандидат предлагает измерять: `go test -bench -benchmem`, pprof, allocation profiles. Он также учитывает читаемость и ownership, а не только micro-optimization.

26. Как диагностировать лишние heap allocations?

Ожидаемый ответ: использовать несколько инструментов вместе: `go test -bench -benchmem` для количества allocations в benchmark, `go build -gcflags=-m` для просмотра escape decisions, heap profile в pprof для production-like нагрузки, allocation profile для горячих мест, runtime metrics для GC pressure.

Подробное объяснение: `-gcflags=-m` показывает, почему compiler решил переместить значение в heap, но это не говорит автоматически, что проблема важна. Benchmark показывает стоимость конкретного куска кода, а pprof показывает impact на реальной нагрузке. Нужно связывать allocations с latency, CPU, GC pause/time и throughput.

Типичные источники лишних allocations:

- частые конвертации `string <-> []byte`;
- `fmt.Sprintf` в горячем path вместо более дешевой сборки строки;
- создание временных slices/maps без переиспользования там, где это безопасно;

- boxing в interface;
- closures, которые захватывают переменные;
- JSON encoding/decoding больших структур;
- логирование с тяжелой подготовкой полей даже при выключенном уровне.

Что важно услышать: кандидат не должен обещать "убрать все allocations". Цель - убрать значимые allocations в горячих местах, не ухудшив корректность и читаемость. Иногда allocation дешевле, чем сложный object pool с риском data races и reuse bugs.

17.2 Concurrency и context

27. Чем goroutine отличается от OS thread?

Ожидаемый ответ: goroutine - легковесная единица выполнения, управляемая Go runtime scheduler. Многие goroutines мультиплексируются на меньшее число OS threads. Goroutine стартует с небольшим stack, который может расти. OS thread тяжелее и управляется ОС.

Сильный ответ: кандидат понимает, что легковесность не означает бесплатность: goroutines потребляют память, scheduler time и могут создавать нагрузку на внешние ресурсы.

28. Что приводит к goroutine leak?

Ожидаемый ответ: goroutine leak возникает, когда goroutine больше не нужна, но не может завершиться. Частые причины: ожидание чтения/записи в channel без получателя/отправителя, отсутствие реакции на context cancellation, бесконечный retry-loop, зависший внешний вызов без timeout.

Как предотвращать: передавать context, закрывать каналы с понятным ownership, использовать timeouts, bounded queues, WaitGroup, errgroup, проверять lifecycle в тестах.

29. Как обнаружить goroutine leak в production?

Ожидаемый ответ: смотреть рост числа goroutines через runtime metrics, pprof goroutine dump, heap/profile, traces, correlating с RPS и ошибками. Если число goroutines растет после завершения нагрузки и не возвращается, это сигнал leak.

Сильный ответ: кандидат умеет читать goroutine stack traces и связывать их с конкретным blocking point.

30. Когда использовать channel, а когда mutex?

Ожидаемый ответ: mutex хорошо подходит для защиты shared state и коротких критических секций. Channel подходит для передачи ownership, coordination, pipeline, fan-out/fan-in, сигналов завершения. Не стоит использовать channel только потому, что это Go, если обычный mutex проще.

Критерий: если задача - "защитить map", чаще нужен mutex. Если задача - "передать работу worker-ам и собрать результат", channel подходит естественно.

31. Что происходит при чтении из закрытого channel?

Ожидаемый ответ: чтение из закрытого channel немедленно возвращает zero value типа элемента. В comma-ok форме `v, ok := <- ch`, `ok` будет false, когда channel закрыт и буфер пуст.

Деталь: если buffered channel закрыли, сначала будут прочитаны оставшиеся элементы, затем начнут возвращаться zero value и `ok=false`.

32. Что происходит при записи в закрытый channel?

Ожидаемый ответ: запись в закрытый channel вызывает panic. Поэтому закрывать channel должен тот, кто владеет отправкой, обычно sender или координатор sender-ов.

Красный флаг: кандидат предлагает закрывать channel со стороны receiver без гарантии, что никто больше не пишет.

33. Для чего нужен select с default?

Ожидаемый ответ: default делает select неблокирующим. Это полезно для попытки отправки/получения без ожидания, polling, drop policy при переполненной очереди. Но в цикле `for { select { default: } }` можно получить busy loop и загрузить CPU.

Хорошая практика: если нужен цикл, часто добавляют blocking case, timer/ticker или backoff.

34. Как реализовать fan-out/fan-in?

Ожидаемый ответ: fan-out: несколько worker goroutines читают задачи из общего input channel.

Fan-in: результаты нескольких worker-ов собираются в один output channel. Нужны WaitGroup для закрытия output после завершения всех worker-ов и context для отмены.

Сильный ответ: кандидат учитывает backpressure через bounded channels и не забывает закрыть output только после завершения всех отправителей.

35. Как ограничить количество одновременно выполняющихся задач?

Ожидаемый ответ: использовать worker pool фиксированного размера, semaphore на buffered channel, errgroup с limit, или rate limiter. Выбор зависит от того, есть ли очередь задач, нужны ли результаты и как обрабатывается отмена.

Пример: buffered channel `sem := make(chan struct{}, limit)`; перед запуском работы отправляем token, после завершения освобождаем token через defer.

36. Как корректно остановить worker pool?

Ожидаемый ответ: закрыть input channel, чтобы worker-ы завершили range; или отменить context, чтобы worker-ы прекратили ожидание/работу. Координатор ждет worker-ы через WaitGroup. Output channel закрывается после завершения всех worker-ов.

Сильный ответ: кандидат объясняет, как избежать отправки в закрытый channel и как завершить обработку частичных результатов.

37. Как работает context.Context и что нельзя хранить в context?

Ожидаемый ответ: context переносит deadline, cancellation signal и request-scoped values через границы API. Его нужно передавать первым параметром функции. В context не хранят optional parameters, большие объекты, mutable state, database handles, logger как обязательную зависимость, если это ломает явный контракт.

Практика: context values подходят для request id, trace id, auth claims, если они действительно request-scoped.

38. Почему нельзя игнорировать ctx.Done() в долгих операциях?

Ожидаемый ответ: если операция не слушает context, она продолжит потреблять ресурсы после отмены запроса, timeout клиента или shutdown. Это ведет к goroutine leaks, лишней нагрузке и медленному завершению сервиса.

Пример: worker, который делает HTTP requests, должен создавать request через `http.NewRequestWithContext`.

39. Что такое data race?

Ожидаемый ответ: data race возникает, когда две goroutines одновременно обращаются к одной памяти, хотя бы одна операция - запись, и между ними нет synchronization. Поведение программы становится неопределенным с точки зрения Go memory model.

Как искать: `go test -race`, code review, pprof/block/mutex profiles. Race detector помогает, но проверяет только выполненные paths, поэтому не доказывает отсутствие races.

40. Когда использовать `sync/atomic` вместо `mutex`?

Ожидаемый ответ: `atomic` подходит для простых независимых операций над числами/указателями: `counters`, `flags`, `copy-on-write pointers`. `Mutex` лучше для сложных инвариантов, нескольких полей, `map/slice`, операций `read-modify-write` над структурой.

Риск: `atomic`-код легко сделать формально `race-free`, но логически неверным. Для Middle+ важно выбирать читаемость и корректность.

41. Что такое `backpressure`?

Ожидаемый ответ: `backpressure` - механизм, который не дает `producer`-ам бесконечно генерировать работу быстрее, чем `consumers` могут ее обработать. Реализация: `bounded queues`, `rate limiting`, `semaphore`, отказ с 429/503, `drop policy`, `circuit breaker`.

Сильный ответ: кандидат связывает `backpressure` с защитой памяти, внешних зависимостей и `latency`.

42. Как сделать `graceful shutdown` HTTP-сервера?

Ожидаемый ответ: поймать `SIGTERM/SIGINT`, перестать принимать новые запросы через `http.Server.Shutdown(ctx)`, дать текущим запросам `deadline` на завершение, остановить `background workers`, закрыть подключения к БД/очередям после завершения работы, экспортировать `readiness=false` до остановки.

Важная деталь: `Shutdown` использует `context` с `timeout`. Если `timeout` истек, соединения закрываются принудительно.

17.3 Backend, API и данные

43. Как проектировать REST API для нового ресурса?

Ожидаемый ответ: начать с доменной модели и операций над ресурсами, определить URL, методы, `request/response schemas`, `status codes`, `validation errors`, `idempotency`, `pagination`, `authorization`, `versioning`. API должен быть стабильным контрактом, а не отражением внутренней структуры БД.

Пример: `GET /users/{id}`, `POST /users`, `PATCH /users/{id}`. Ошибки возвращаются в едином формате с `machine-readable code` и `human-readable message`.

44. Какие HTTP-коды использовать для типовых ошибок?

Ожидаемый ответ: 400 `Bad Request` для синтаксически неверного запроса, 422 `Unprocessable Entity` для валидного JSON с доменной `validation error`, 401 `Unauthorized` для отсутствующей/невалидной аутентификации, 403 `Forbidden` для запрета при известной личности, 404 `Not Found`, 409 `Conflict`, 429 `Too Many Requests`, 500 для `unexpected server errors`, 503 для временной недоступности.

Сильный ответ: кандидат не раскрывает `internal error` клиенту, но логирует `details` с `correlation id`.

45. Где должна жить бизнес-логика: `handler`, `service` или `repository`?

Ожидаемый ответ: `handler` отвечает за `transport`: `parsing`, `validation` на уровне формата, `auth context`, `mapping errors to HTTP`. `Service` содержит бизнес-правила и транзакционные `use cases`. `Repository` отвечает за `persistence` и `SQL/DB details`.

Практический критерий: бизнес-правило должно тестироваться без HTTP и желательно без реальной БД, если это не `integration test`.

46. Как реализовать `idempotency` для POST-запроса?

Ожидаемый ответ: клиент отправляет `idempotency key`. Сервер сохраняет ключ, `fingerprint` запроса, статус выполнения и результат. Повторный запрос с тем же ключом и тем же `fingerprint` возвращает тот же результат. Если `fingerprint` отличается - ошибка `conflict`.

Важные детали: атомарное сохранение ключа и результата, TTL для ключей, защита от гонок, учет пользователя/tenant в scope ключа.

47. Когда выбирать gRPC вместо REST?

Ожидаемый ответ: gRPC хорош для internal service-to-service communication, строгих контрактов protobuf, streaming, низкой latency, code generation, polyglot environments. REST проще для публичных API, браузеров, дебага через curl и широкого ecosystem.

Сильный ответ: выбор зависит от клиентов, инфраструктуры, observability, backward compatibility и требований к streaming.

48. Что такое транзакция?

Ожидаемый ответ: транзакция - группа операций с ACID-свойствами: atomicity, consistency, isolation, durability. commit фиксирует изменения, rollback отменяет их. Транзакция нужна, когда несколько изменений должны произойти как единое целое.

Практика в Go: context-aware BeginTx, defer rollback, явный commit, аккуратная обработка ошибки commit.

49. Какие уровни изоляции транзакций вы знаете?

Ожидаемый ответ: Read Uncommitted, Read Committed, Repeatable Read, Serializable. В PostgreSQL Read Uncommitted фактически работает как Read Committed. Более высокий уровень уменьшает аномалии, но может увеличить блокировки, serialization failures и стоимость.

Сильный ответ: кандидат говорит не только названия, но и какие аномалии предотвращаются и как это влияет на retry транзакций.

50. Что такое dirty read, non-repeatable read, phantom read?

Ожидаемый ответ: dirty read - чтение незакоммиченных данных другой транзакции.

Non-repeatable read - повторное чтение той же строки в одной транзакции дает другой результат из-за commit другой транзакции. Phantom read - повторный запрос по условию возвращает другой набор строк из-за вставок/удалений другой транзакции.

51. Как избежать lost update?

Ожидаемый ответ: использовать row-level locks (SELECT ... FOR UPDATE), atomic update with condition (UPDATE accounts SET balance=balance-? WHERE id=? AND balance>=?), optimistic locking через version column, или serializable transaction с retry.

Сильный ответ: кандидат учитывает deadlocks, порядок блокировок и retry policy.

52. Как работает optimistic locking?

Ожидаемый ответ: у записи есть версия или updated_at. При обновлении запрос включает старую версию: UPDATE table SET ..., version=version+1 WHERE id=? AND version=?. Если affected rows = 0, значит запись изменилась конкурентно, нужно повторить чтение/операцию или вернуть conflict.

53. Как выбрать индекс для SQL-запроса?

Ожидаемый ответ: смотреть WHERE, JOIN, ORDER BY, GROUP BY, selectivity, cardinality и реальные query plans через EXPLAIN ANALYZE. Композитный индекс должен учитывать порядок колонок и pattern запроса.

Важная деталь: индекс ускоряет чтение, но замедляет записи и занимает место. Индексы нужны под реальные запросы, а не "на всякий случай".

54. Почему индекс может не использоваться?

Ожидаемый ответ: низкая selectivity, функция над индексируемой колонкой, несовпадение типов, leading wildcard в LIKE, устаревшая статистика, planner считает seq scan дешевле, неправильный порядок колонок в composite index.

55. Что такое N+1 problem и как исправлять?

Ожидаемый ответ: N+1 возникает, когда после одного запроса списка для каждого элемента делается отдельный запрос деталей. Исправление: JOIN, batch query `WHERE id IN (...)`, preloading, dataloader pattern, изменение API/агрегации.

Сильный ответ: кандидат думает о лимитах batch size и размере результата.

56. Как проектировать миграции без downtime?

Ожидаемый ответ: использовать expand/contract pattern. Сначала добавить backward-compatible schema changes, затем задеплоить код, который пишет/читает новое поле, выполнить backfill батчами, переключить чтение, потом удалить старое поле отдельным релизом.

Риски: долгие locks, default values на больших таблицах, backfill без throttling, несовместимость старой и новой версии сервиса.

57. Как хранить nullable-поля в Go?

Ожидаемый ответ: варианты: `sql.NullString` и аналоги, pointer fields, custom nullable-типы, nullable-типы ORM/driver. Выбор зависит от различия между "отсутствует", "null" и zero value.

Сильный ответ: кандидат учитывает JSON serialization и доменную модель, а не просто технический тип БД.

58. Offset vs cursor pagination?

Ожидаемый ответ: offset pagination проста, но на больших данных становится медленной и нестабильной при вставках/удалениях. Cursor pagination использует stable sort key и cursor, лучше для больших списков и realtime changes, но сложнее для произвольного перехода на страницу.

Практика: нужен deterministic ordering, например `(created_at, id)`, чтобы избежать пропусков и дублей.

17.4 Reliability и внешние зависимости

59. Что такое timeout, retry и circuit breaker?

Ожидаемый ответ: timeout ограничивает время ожидания операции. Retry повторяет временно неуспешную операцию. Circuit breaker временно прекращает вызовы к нестабильной зависимости, чтобы дать ей восстановиться и защитить свой сервис.

Сильный ответ: эти механизмы должны использоваться вместе с idempotency, backoff, jitter, retry budget и observability.

60. Почему retries могут ухудшить инцидент?

Ожидаемый ответ: retries увеличивают нагрузку на уже деградирующую зависимость, могут вызвать retry storm, дублировать side effects и увеличить latency. Без backoff и лимитов retries часто превращают частичный сбой в полный.

Правильный подход: retry только для retryable errors, ограниченное число попыток, exponential backoff with jitter, timeout на всю операцию, idempotency key для side-effect операций.

61. Что такое partial failure?

Ожидаемый ответ: ситуация, когда часть операции успешна, а часть нет: один downstream ответил, другой нет; запись в БД прошла, публикация события нет; часть batch обработана. Distributed systems должны проектироваться с учетом partial failure.

Паттерны: transactions там, где возможно, outbox, sagas, compensating actions, idempotent consumers, reconciliation jobs.

62. Как проектировать взаимодействие с внешним API?

Ожидаемый ответ: явные timeouts, retries с backoff, circuit breaker, rate limiting, idempotency для side effects, typed client, contract tests, обработка статусов, structured logging без секретов, metrics по latency/errors, fallback или graceful degradation.

17.5 Практические задачи: ожидаемые решения

63. In-memory rate limiter: какой алгоритм выбрать?

Ожидаемый ответ: для простой задачи подойдет fixed window, но у него есть burst на границе окна. Sliding window точнее, но сложнее и дороже. Token bucket хорошо контролирует среднюю скорость и допускает ограниченный burst. Для API часто выбирают token bucket или sliding window в зависимости от требований.

Корректная реализация: хранить состояние по userID, защищать map mutex-ом или шардировать, использовать injectable clock для тестов, периодически очищать неактивных пользователей, не держать lock во время долгих операций.

Ограничение: in-memory limiter не работает корректно при нескольких инстансах без sticky sessions или внешнего shared storage вроде Redis.

64. Concurrent fetch with timeout: как должна выглядеть архитектура?

Ожидаемый ответ: создать context с общим timeout или принять внешний context, ограничить параллелизм worker pool/semaphore, каждый request создавать через `http.NewRequestWithContext`, результаты отправлять в channel или защищенную структуру, дожидаться worker-ов, закрыть result channel, вернуть частичные результаты и агрегированную ошибку.

Важные edge cases: `limit <= 0`, пустой список URL, отмена context до старта, зависший сервер, не-2xx статус, сохранение порядка результатов, отсутствие goroutine leaks.

65. Transactional money transfer: как решить безопасно?

Ожидаемый ответ: начать транзакцию на уровне service/use case, заблокировать счета в детерминированном порядке по id через `SELECT ... FOR UPDATE`, проверить существование и баланс, выполнить debit/credit, записать audit log, commit. Доменные ошибки отделить от инфраструктурных.

Сильные детали: idempotency key для повторного запроса клиента, retry при deadlock/serialization failure, decimal/integer minor units вместо float, audit log в той же транзакции, outbox для события после commit.

17.6 Тестирование

66. Что такое table-driven tests?

Ожидаемый ответ: тесты, где набор кейсов описан таблицей структур, а один test loop прогоняет функцию на каждом кейсе. Это снижает дублирование и делает edge cases явными.

Хорошая практика: использовать `t.Run(name, func(t *testing.T) {...})`, не забывать capture переменной в старых версиях Go/при параллельных тестах, проверять и happy path, и ошибки.

67. Как тестировать код, зависящий от времени?

Ожидаемый ответ: не вызывать `time.Now()` глубоко в бизнес-логике напрямую. Передавать clock interface/function, использовать fixed time в тестах, контролируемый fake clock для timers/tickers.

Риск: тесты со sleep обычно flaky и медленные.

68. Как тестировать HTTP handler?

Ожидаемый ответ: использовать `httptest.NewRequest`, `httptest.ResponseRecorder`, подменять service dependency mock/fake-ом, проверять status code, headers, body, error mapping. Для интеграционного уровня можно поднимать `httptest.Server`.

69. Как тестировать интеграцию с БД?

Ожидаемый ответ: использовать отдельную тестовую БД, миграции, transaction rollback per test или очистку данных, testcontainers/docker при возможности, fixtures/factories, проверять реальные constraints и SQL behavior.

Сильный ответ: кандидат разделяет unit tests repository mocks и integration tests с реальной БД.

70. Когда использовать mocks, fakes и testcontainers?

Ожидаемый ответ: mocks подходят для проверки взаимодействий на границах, но ими легко закрепить implementation details. Fakes часто лучше для доменной логики. Testcontainers/реальная инфраструктура нужны, когда важно поведение драйвера, SQL, брокера, транзакций.

17.7 Performance и profiling

71. Как понять, где bottleneck: CPU, память, БД или сеть?

Ожидаемый ответ: начать с метрик: latency breakdown, CPU, memory, GC, DB query duration, connection pool stats, network errors. Затем использовать pprof, tracing, logs с correlation id. Без измерений оптимизация будет гаданием.

72. Как использовать pprof?

Ожидаемый ответ: подключить `net/http/pprof` или собрать профиль программно, снять `cpu/heap/goroutine/mutex/block` profiles, открыть через `go tool pprof`, смотреть top, list, web/flamegraph. Профилировать нужно под репрезентативной нагрузкой.

73. Что может привести к высокому GC pressure?

Ожидаемый ответ: большое число short-lived allocations, лишние conversions `[]byte/string`, создание объектов в горячих циклах, неограниченные buffers, крупные временные структуры, чрезмерное логирование/JSON encoding.

Исправление: измерить allocations benchmark-ами и heap profile, переиспользовать buffers осторожно, уменьшить промежуточные объекты, stream processing вместо загрузки всего в память.

74. Как искать memory leak в Go?

Ожидаемый ответ: в Go leak часто означает удержание ссылок или goroutine leak. Сравнить heap profiles во времени, смотреть goroutine dump, проверять caches без eviction, slices, которые держат большой backing array, global maps, channels/workers.

17.8 Observability и эксплуатация

75. Какие structured logs нужны HTTP-сервису?

Ожидаемый ответ: request id/trace id, method, path/route pattern, status, latency, user/tenant id при допустимости, error code, dependency name/status/latency для downstream errors. Нельзя логировать пароли, токены, персональные данные без необходимости.

76. Какие RED/USE-метрики вы знаете?

Ожидаемый ответ: RED для сервисов: Rate, Errors, Duration. USE для ресурсов: Utilization, Saturation, Errors. Для HTTP нужны RPS, error rate, latency percentiles, in-flight requests. Для БД - pool

usage, query latency, errors.

77. Readiness vs liveness?

Ожидаемый ответ: liveness отвечает, жив ли процесс и нужно ли его перезапустить. Readiness отвечает, готов ли инстанс принимать трафик. При shutdown readiness должна стать false до остановки обработки.

Риск: если liveness зависит от временной недоступности БД, Kubernetes может начать перезапускать здоровые, но временно деградировавшие поды.

17.9 Security

78. Как хранить пароли пользователей?

Ожидаемый ответ: никогда не хранить plaintext. Использовать password hashing algorithms с солью и work factor: bcrypt, scrypt, Argon2id. Соль должна быть уникальной, pepper можно хранить отдельно в secret manager. Нужно поддерживать rehash при изменении параметров.

79. Authentication vs authorization?

Ожидаемый ответ: authentication подтверждает, кто пользователь. Authorization решает, что этому пользователю разрешено. Пользователь может быть аутентифицирован, но не иметь доступа к ресурсу.

80. Как устроен JWT и какие риски?

Ожидаемый ответ: JWT состоит из header, payload claims и signature. Риски: long-lived tokens, невозможность простого revoke без state, неправильная проверка alg/audience/issuer/expiration, хранение секретных данных в payload, утечка токена.

Практика: короткий TTL access token, refresh flow, проверка iss, aud, exp, key rotation, HTTPS, безопасное хранение на клиенте.

81. Как защищаться от SQL injection?

Ожидаемый ответ: использовать parameterized queries/prepared statements, не конкатенировать user input в SQL, валидировать identifiers отдельно через allowlist, ограничивать права DB user. ORM не гарантирует защиту, если разработчик вставляет raw SQL небезопасно.

17.10 Code review

82. Как ревьюить PR с business logic в handler?

Ожидаемый ответ: указать, что handler смешивает transport и доменную логику, из-за чего код сложнее тестировать и переиспользовать. Предложить вынести use case в service, оставить в handler parsing/auth/error mapping. Попросить unit tests на service и handler tests на HTTP mapping.

Хороший комментарий: "Давай вынесем расчет лимита в service: так мы сможем протестировать бизнес-правила без HTTP и не будем дублировать их при появлении gRPC endpoint".

83. Как приоритизировать замечания в code review?

Ожидаемый ответ: сначала correctness, security, data loss, race conditions, observability, backward compatibility. Затем maintainability и тестируемость. Стиль и вкусовые предпочтения должны быть автоматизированы линтерами или помечены как non-blocking.

17.11 System design: сервис уведомлений

84. Какой базовый дизайн предложить?

Ожидаемый ответ: API принимает событие, валидирует его и сохраняет durable record. Через outbox или транзакционную публикацию событие попадает в брокер. Worker-ы отправляют email/SMS/push через provider clients. Статусы и попытки сохраняются в БД. Для retry используется delayed queue/backoff, для исчерпанных попыток - DLQ.

Компоненты: ingress API, DB, outbox publisher, message broker, workers per channel/provider, template service, status storage, metrics/logs/traces, admin/reconciliation tooling.

85. Как не отправить уведомление дважды?

Ожидаемый ответ: использовать idempotency key/event id, unique constraint на (tenant_id, event_id, channel), idempotent consumer, хранить provider message id, дедуплицировать retries. При at-least-once доставке нужно проектировать consumers как idempotent.

86. Как гарантировать, что событие не потеряется?

Ожидаемый ответ: сохранить событие в durable storage в рамках транзакции с outbox record. Отдельный publisher читает outbox и публикует в брокер, пометая опубликованное. Если publisher падает, он продолжает с неопубликованных записей. Брокер должен подтверждать запись, consumer - ack после успешной обработки.

Сильный ответ: кандидат признает, что exactly-once в распределенных системах обычно заменяется at-least-once + idempotency.

87. Что делать, если SMS-провайдер недоступен 30 минут?

Ожидаемый ответ: circuit breaker открывается, retries идут с backoff и jitter, задачи остаются в очереди или delayed retry storage, критичные уведомления можно переключить на fallback provider, некритичные деградировать. Нужны alerts по error rate, queue lag, delivery latency.

88. Какие метрики нужны сервису уведомлений?

Ожидаемый ответ: accepted events rate, validation errors, queue lag, send attempts, success/failure by channel/provider, retry count, DLQ size, delivery latency percentiles, provider latency/status, template rendering errors, worker saturation.

17.12 Поведенческий блок: что считать сильным ответом

89. Кандидат не согласился с техническим решением команды. Что слушать в ответе?

Сильный ответ: кандидат сначала понял контекст и ограничения, сформулировал риски, предложил альтернативы, обсудил trade-off, принял командное решение после дискуссии и помог его реализовать. Важно отсутствие позиции "я один был прав, остальные не понимали".

90. Кандидат ошибся в production. Что слушать?

Сильный ответ: конкретное описание инцидента, impact, timeline, mitigation, root cause, follow-up actions. Кандидат не перекладывает вину, говорит о системных улучшениях: tests, alerts, rollout strategy, runbook, feature flag.

91. Как кандидат выбирает между quick fix и refactoring?

Сильный ответ: оценивает impact, срочность, риск регрессии, blast radius, наличие тестов. В инциденте сначала безопасная стабилизация, затем отдельный follow-up на системное исправление. Если quick fix создает долг, он должен быть видимым и запланированным.

92. Как кандидат помогает менее опытному разработчику?

Сильный ответ: задает вопросы, объясняет reasoning, дает небольшие самостоятельные задачи, делает pairing, пишет понятные review comments, делится контекстом. Не забирает задачу полностью и не унижает за ошибки.

17.13 Расширенный блок по базам данных

Этот блок стоит использовать, если роль предполагает самостоятельную работу с PostgreSQL/MySQL, проектирование схемы, оптимизацию запросов, миграции и разбор production-проблем с данными. Вопросы можно задавать выборочно: для Middle+ важно не знать все названия наизусть, а уметь объяснять причинно-следственные связи и выбирать безопасное решение.

93. Что происходит внутри БД при выполнении SQL-запроса?

Ожидаемый ответ: в общем виде запрос проходит parsing, semantic analysis, rewrite, planning/optimization и execution. Planner выбирает план на основе статистики, доступных индексов, оценочной стоимости чтения страниц, фильтрации, join strategy и сортировок. Executor выполняет выбранный план и возвращает строки клиенту.

Объяснение: важно, что БД не "просто использует индекс, если он есть". Она сравнивает стоимость разных вариантов. Если таблица маленькая или условие выбирает большую долю строк, sequential scan может быть дешевле index scan. Поэтому оптимизация начинается с EXPLAIN/EXPLAIN ANALYZE, а не с добавления случайных индексов.

Сильный ответ: кандидат упоминает статистику, cardinality/selectivity, join algorithms, сортировки и то, что план может измениться после роста данных.

94. Чем EXPLAIN отличается от EXPLAIN ANALYZE?

Ожидаемый ответ: EXPLAIN показывает предполагаемый план и оценочные стоимости без выполнения запроса. EXPLAIN ANALYZE реально выполняет запрос и показывает фактическое время, количество строк, loops и расхождения между оценками и реальностью.

Объяснение: если estimates сильно отличаются от actual rows, planner может выбрать плохой join или scan. Причиной могут быть устаревшие statistics, коррелированные колонки, неудачный индекс или неравномерное распределение данных.

Осторожность: EXPLAIN ANALYZE для INSERT/UPDATE/DELETE выполняет изменение. В production его нужно использовать аккуратно, иногда внутри rollback-транзакции или на replica/staging.

95. Какие поля в EXPLAIN ANALYZE вы смотрите первыми?

Ожидаемый ответ: actual time, actual rows, loops, разницу estimated rows vs actual rows, тип scan, join type, sort method, heap fetches, buffers при BUFFERS, filter rows removed, наличие temporary files.

Объяснение: медленный запрос часто не в том месте, где самая большая cost estimate, а там, где много loops или резко больше actual rows. Например, nested loop может быть нормальным для десятков строк, но разрушительным для сотен тысяч.

Сильный ответ: кандидат умеет объяснить, почему один node с loops=10000 может давать основную задержку даже при маленьком времени одного loop.

96. Что такое selectivity и cardinality индекса?

Ожидаемый ответ: cardinality - количество уникальных значений в колонке или наборе колонок. Selectivity - насколько условие фильтрует данные: какую долю строк оно оставляет. Чем выше selectivity условия, тем полезнее индекс.

Объяснение: индекс по boolean-полю часто бесполезен, если значения распределены 50/50 и запрос выбирает большую часть таблицы. Но partial index по редкому состоянию, например WHERE status = 'pending', может быть очень полезен.

Пример: для таблицы заказов индекс по user_id обычно полезен, если запросы выбирают заказы одного пользователя из миллионов строк. Индекс по is_deleted, где почти все строки false, часто не помогает без partial condition.

97. Как работает composite index и почему важен порядок колонок?

Ожидаемый ответ: composite index строится по нескольким колонкам в заданном порядке. Для B-tree эффективно используется leftmost prefix: индекс (tenant_id, created_at, id) хорошо подходит для фильтра по tenant_id и сортировки/диапазона по created_at, но не так полезен для запроса только по created_at без tenant_id.

Объяснение: первая колонка должна обычно соответствовать equality/filter с хорошим сужением или tenant boundary, затем идут колонки диапазона и сортировки. Но порядок зависит от конкретных запросов, распределения данных и условий.

Сильный ответ: кандидат не отвечает универсальным "самую селективную колонку первой" без контекста. Для multi-tenant систем tenant_id часто первым нужен для изоляции данных и pruning, даже если другая колонка более селективна глобально.

98. Чем B-tree, Hash, GIN и GiST индексы отличаются на практике?

Ожидаемый ответ: B-tree - основной индекс для equality, range, ordering. Hash - для equality, но в PostgreSQL используется реже, потому что B-tree обычно достаточно универсален. GIN полезен для массивов, JSONB, full-text search, containment. GiST полезен для геоданных, ranges, similarity и некоторых custom operators.

Объяснение: тип индекса должен соответствовать оператору в запросе. Индекс не магический: JSONB containment @> может использовать GIN, но произвольная функция над JSON может не использовать индекс без expression index.

Пример: WHERE data @> '{"role":"admin"}' может выиграть от GIN по JSONB, а WHERE lower(email)=... требует либо хранения нормализованного email, либо expression index на lower(email).

99. Что такое covering index / index-only scan?

Ожидаемый ответ: index-only scan возможен, когда БД может ответить на запрос, используя только индекс, без чтения строк таблицы. Для этого нужные колонки должны быть в индексе, а visibility information должна позволять не ходить в heap.

Объяснение: в PostgreSQL даже если все колонки есть в индексе, visibility map влияет на то, можно ли избежать heap fetches. Частые обновления могут снижать пользу index-only scan.

Практический вывод: INCLUDE-колонки могут помочь покрыть запрос, но каждый дополнительный индекс и каждая включенная колонка увеличивают размер индекса и стоимость записи.

100. Что такое partial index и когда он полезен?

Ожидаемый ответ: partial index индексирует только строки, удовлетворяющие условию. Он полезен, когда запросы часто работают с маленьким подмножеством данных: активные записи, pending jobs, не удаленные сущности, конкретный tenant tier.

Пример: CREATE INDEX ON jobs (run_at) WHERE status = 'pending'; помогает worker-ам быстро находить готовые задачи, не индексируя завершённые jobs.

Объяснение: partial index меньше, быстрее обновляется и лучше помещается в cache. Но planner использует его только если условие запроса логически соответствует predicate индекса.

101. Что такое expression index?

Ожидаемый ответ: expression index строится по выражению, а не по сырой колонке: например CREATE INDEX ON users (lower(email));. Он полезен, когда запрос фильтрует или сортирует по вычисляемому выражению.

Объяснение: если написать WHERE lower(email) = lower(\$1), обычный индекс по email обычно не поможет, потому что функция применяется к каждой строке. Expression index позволяет хранить вычисленное значение в индексе.

Риск: expression должен совпадать с запросом, а функции должны быть подходящими для индексации. Также индекс увеличивает стоимость записи.

102. Почему слишком много индексов - проблема?

Ожидаемый ответ: каждый индекс занимает место, должен обновляться при insert/update/delete, увеличивает WAL/redo log, может замедлять bulk operations, усложняет planning и повышает требования к cache.

Объяснение: индекс ускоряет конкретные чтения, но является дополнительной структурой записи. Если добавить индекс "на всякий случай", можно улучшить один редкий запрос и ухудшить частые записи.

Сильный ответ: кандидат предлагает отслеживать unused indexes, slow queries, write amplification и удалять индексы после проверки.

103. Что такое transaction isolation anomaly на примере lost update?

Ожидаемый ответ: lost update возникает, когда две транзакции читают одно значение, независимо считают новое и записывают результат, из-за чего одно обновление затирает другое.

Пример: два запроса одновременно читают баланс 100. Первый списывает 10 и пишет 90. Второй списывает 20, но тоже исходил из 100 и пишет 80. Итог должен быть 70, но получилось 80.

Как избежать: atomic update с условием, row lock FOR UPDATE, optimistic locking по version, serializable isolation с retry. Просто обернуть операции в транзакцию недостаточно, если изоляция и locking не предотвращают гонку.

104. Что такое write skew?

Ожидаемый ответ: write skew - аномалия, когда две транзакции читают общий набор данных, принимают решение, а затем обновляют разные строки так, что нарушается инвариант, хотя прямого конфликта записи в одну строку нет.

Пример: в расписании всегда должен быть хотя бы один дежурный врач. Две транзакции одновременно видят двух врачей на смене и каждая снимает одного разного врача. Обе обновляют разные строки, но итог - ни одного дежурного.

Как избежать: serializable isolation, explicit locks на набор/таблицу/агрегат, материализованный invariant row, constraints там, где возможно. Middle+ должен понимать, что row-level lock одной строки не всегда защищает инвариант над набором строк.

105. Что такое phantom read и почему он важен для бизнес-инвариантов?

Ожидаемый ответ: phantom read возникает, когда повторный запрос по условию внутри транзакции видит новый набор строк из-за вставки/удаления другой транзакции. Это важно, если бизнес-правило зависит от количества или отсутствия строк.

Пример: "у пользователя может быть только один активный subscription". Две транзакции одновременно проверяют, что активной подписки нет, и обе вставляют новую. Без unique constraint или serializable/locking получится нарушение.

Практический вывод: инварианты лучше закреплять constraints/unique indexes, а не только проверками в коде.

106. Чем pessimistic locking отличается от optimistic locking?

Ожидаемый ответ: pessimistic locking блокирует данные до изменения, предполагая высокий риск конфликта: SELECT ... FOR UPDATE. Optimistic locking не блокирует заранее, а проверяет версию при записи, предполагая, что конфликты редки.

Объяснение: pessimistic locking проще для критичных операций, но может снижать concurrency и приводить к deadlocks. Optimistic locking лучше масштабируется при редких конфликтах, но требует retry/конфликтного ответа клиенту.

Критерий выбора: высокая конкуренция за одни и те же строки и строгие деньги/остатки - часто pessimistic или atomic update. Редактирование профиля/документа - часто optimistic version.

107. Что делает `SELECT ... FOR UPDATE?`

Ожидаемый ответ: он читает строки и ставит row-level lock, запрещая другим транзакциям изменить или заблокировать эти строки несовместимым lock до commit/rollback текущей транзакции.

Объяснение: lock действует только на найденные строки. Если нужно защитить отсутствие строки или диапазон, могут понадобиться unique constraints, higher isolation, predicate locks или explicit locking strategy.

Практика: для перевода денег блокируют оба счета в стабильном порядке, например по возрастанию id, чтобы снизить риск deadlock.

108. Что такое deadlock в БД и как его предотвращать?

Ожидаемый ответ: deadlock возникает, когда транзакции ждут locks друг друга по циклу. БД обычно обнаруживает deadlock, прерывает одну транзакцию, и приложение должно корректно обработать ошибку.

Профилактика: брать locks в стабильном порядке, держать транзакции короткими, не делать внешние HTTP-вызовы внутри транзакции, индексировать условия update/delete, избегать интерактивных долгих транзакций.

Сильный ответ: кандидат говорит о retry только для безопасных транзакций и с ограничением попыток/backoff.

109. Почему нельзя делать внешние API-вызовы внутри DB-транзакции?

Ожидаемый ответ: внешние вызовы непредсказуемы по latency и могут зависнуть. Пока транзакция открыта, она держит locks, connection из pool, snapshot/версионность и может блокировать другие операции.

Объяснение: это увеличивает contention и риск deadlock/timeouts. Кроме того, если внешний side effect выполнен, а транзакция потом rollback, возникает рассинхронизация.

Лучший подход: сначала зафиксировать состояние и outbox event в транзакции, затем асинхронно вызвать внешний сервис. Для синхронных сценариев нужно очень явно понимать idempotency и компенсации.

110. Как правильно использовать `defer tx.Rollback()` в Go?

Ожидаемый ответ: после `BeginTx` часто делают `defer tx.Rollback()`. Если позже `Commit` успешен, deferred rollback вернет ошибку, которую обычно игнорируют. Это гарантирует rollback на любом early return до commit.

Объяснение: важно не затереть ошибку commit и не считать транзакцию успешной до успешного commit. Ошибка commit может означать, что изменения не зафиксированы или статус неизвестен, в зависимости от драйвера/сети.

Пример паттерна: `begin, defer rollback, операции, commit, return`. В сложном коде лучше иметь helper `WithTx(ctx, func(tx) error)`.

111. Как настроить connection pool в Go и какие симптомы неправильной настройки?

Ожидаемый ответ: в `database/sql` важны `SetMaxOpenConns`, `SetMaxIdleConns`, `SetConnMaxLifetime`, `SetConnMaxIdleTime`. Слишком маленький pool вызывает ожидание connections и рост latency. Слишком большой pool перегружает БД и увеличивает contention.

Объяснение: pool приложения должен соответствовать capacity БД и числу инстансов сервиса. Если 20 pod-ов по 100 connections, БД увидит до 2000 connections, что может быть катастрофой.

Метрики: open connections, in-use, idle, wait count, wait duration, query latency, DB CPU, locks. Middle+ должен связать latency приложения с ожиданием connection pool, а не только с медленными SQL.

112. Что такое prepared statements и parameterized queries?

Ожидаемый ответ: parameterized query отделяет SQL-код от пользовательских значений, защищая от SQL injection и позволяя драйверу/БД безопасно передавать параметры. Prepared statement может быть предварительно разобран/спланирован и переиспользован.

Объяснение: безопасность - главный аргумент. Performance зависит от БД, драйвера и plan caching. Нельзя подставлять user input через string concatenation, особенно в WHERE. Для dynamic identifiers используют allowlist, потому что параметры обычно не заменяют имена таблиц/колонок.

113. Что такое SQL injection в ORDER BY и как от нее защищаться?

Ожидаемый ответ: параметры безопасно подставляют значения, но не имена колонок или направление сортировки. Если пользователь передает `sort=created_at desc; drop table...`, конкатенация в ORDER BY опасна.

Защита: allowlist: map внешних имен сортировки в заранее известные SQL expressions. Direction тоже allowlist ASC/DESC. Не принимать произвольный SQL-fragment от клиента.

114. Почему SELECT * часто плохая идея?

Ожидаемый ответ: SELECT * тянет лишние данные, увеличивает network IO, мешает index-only scans, делает код чувствительным к изменению схемы и может случайно начать читать тяжелые поля.

Объяснение: явный список колонок - это контракт. Он упрощает review, снижает нагрузку и защищает от сюрпризов при добавлении новых колонок вроде JSON/blob.

115. Offset pagination: почему она деградирует?

Ожидаемый ответ: при OFFSET N LIMIT M БД часто должна найти и пропустить N строк, поэтому большой offset становится дорогим. Кроме того, при новых вставках/удалениях между запросами пользователь может увидеть дубли или пропуски.

Лучшее решение: keyset/cursor pagination по стабильному порядку, например (created_at, id). Cursor хранит последнее значение сортировки и id, следующий запрос делает WHERE (created_at, id) < (\$lastCreatedAt, \$lastID).

116. Как проектировать cursor pagination правильно?

Ожидаемый ответ: нужен детерминированный порядок, уникальный tie-breaker, непрозрачный cursor, стабильные фильтры, направление сортировки, обработка deleted/inserted rows. Cursor обычно кодирует значения sort keys, а не номер страницы.

Объяснение: если сортировать только по created_at, несколько строк с одинаковым временем могут пропадать или дублироваться. Поэтому добавляют id как tie-breaker.

Сильный ответ: кандидат учитывает безопасность cursor: не доверять ему слепо, подписывать или валидировать, не раскрывать лишние данные.

117. Как безопасно делать schema migration на большой таблице?

Ожидаемый ответ: избегать долгих exclusive locks. Делать изменения маленькими шагами: добавить nullable column без тяжелого default, задеплоить код с dual write/read fallback, backfill батчами с throttling, добавить constraint/index concurrently, переключить чтение, затем удалить старую колонку отдельным шагом.

Объяснение: опасность не в самой миграции как SQL-файле, а в lock duration, WAL, replication lag, IO и несовместимости старой/новой версии приложения.

Сильный ответ: кандидат спрашивает про конкретную БД и версию, потому что поведение DDL отличается между PostgreSQL/MySQL.

118. Как добавлять NOT NULL колонку с default на большой таблице?

Ожидаемый ответ: зависит от БД и версии. Безопасный общий подход: добавить nullable column, начать писать значение из приложения, backfill старыми строками батчами, добавить constraint/validate, затем сделать колонку NOT NULL.

Объяснение: в некоторых версиях БД добавление column with default может переписать всю таблицу или взять тяжелый lock. Даже если современная версия оптимизирует fast default, нужно проверить поведение.

119. Что такое online index creation?

Ожидаемый ответ: создание индекса без длительной блокировки записей. В PostgreSQL это CREATE INDEX CONCURRENTLY, в MySQL - online DDL в зависимости от engine/version. Оно обычно дольше и имеет ограничения, но снижает downtime.

Объяснение: обычный CREATE INDEX может блокировать writes. Concurrent index build снижает impact, но все равно создает нагрузку на IO/CPU и может усилить replication lag.

120. Что такое replication lag и чем он опасен?

Ожидаемый ответ: replication lag - задержка между primary и replica. Опасность: приложение читает с replica и не видит только что записанные данные, пользователь получает stale read, фоновые процессы принимают решения на старых данных.

Пример: после создания заказа клиент сразу делает GET, который идет на replica, и получает 404. Это read-after-write inconsistency.

Решения: читать критичные read-after-write запросы с primary, sticky reads на время, отслеживать lag, использовать causal consistency tokens, проектировать UI/flow с eventual consistency.

121. Когда читать с replica, а когда с primary?

Ожидаемый ответ: replica подходит для read-heavy аналитических/некритичных запросов, где stale data допустима. Primary нужен для транзакционных reads, read-after-write, проверок инвариантов, authorization/permissions, финансовых операций.

Сильный ответ: кандидат не переносит все чтения на replica ради "масштабирования", а классифицирует запросы по требованиям консистентности.

122. Что такое read/write splitting и какие у него ловушки?

Ожидаемый ответ: read/write splitting направляет writes на primary, reads на replicas. Ловушки: replication lag, транзакции, read-after-write, разные планы/статистика на replica, failover, сложность отладки.

Практика: слой доступа к данным должен явно уметь выбирать consistency level, а не случайно отправлять любой SELECT на replica.

123. Что такое sharding и когда он нужен?

Ожидаемый ответ: sharding - горизонтальное разделение данных по shard key между несколькими БД/кластерами. Он нужен, когда один кластер уже не справляется по объему, throughput, storage, tenant isolation или operational limits.

Объяснение: sharding резко повышает сложность: cross-shard queries, transactions, rebalancing, hot shards, migrations, operational tooling. До sharding обычно стоит оптимизировать схему, индексы, queries, caching, partitioning, read replicas.

124. Как выбрать shard key?

Ожидаемый ответ: shard key должен равномерно распределять нагрузку, соответствовать частым access patterns, минимизировать cross-shard operations и позволять growth/rebalancing. Часто это tenant_id/user_id, но нужно учитывать hot tenants.

Риск: timestamp как shard key может создать hot shard. Слишком мелкий random key может разрушить locality и усложнить запросы по tenant.

125. Чем partitioning отличается от sharding?

Ожидаемый ответ: partitioning делит таблицу внутри одной логической БД/кластера на partitions, обычно прозрачно для приложения. Sharding делит данные между независимыми БД/кластерами, часто требует routing на уровне приложения/инфраструктуры.

Объяснение: partitioning помогает с управлением большими таблицами, pruning, retention, индексами по частям. Но он не убирает все ограничения одного кластера и не всегда увеличивает write capacity линейно.

126. Когда полезно partitioning по времени?

Ожидаемый ответ: для event/log/audit/time-series таблиц, где большинство запросов ограничено периодом, а старые данные нужно архивировать или удалять. Можно быстро drop old partition вместо массового delete.

Риск: если запросы не фильтруют по partition key, pruning не сработает и придется сканировать много partitions. Слишком много partitions тоже ухудшает planning/maintenance.

127. Как проектировать soft delete?

Ожидаемый ответ: soft delete обычно добавляет deleted_at или status. Нужно решить, участвуют ли удаленные записи в unique constraints, запросах, audit, восстановлении и retention. Часто нужны partial unique indexes, например уникальный email только для deleted_at IS NULL.

Объяснение: soft delete усложняет каждый запрос: нужно не забывать фильтр. Ошибка может показать удаленные данные пользователю. Иногда лучше hard delete + audit/archive, в зависимости от домена и требований.

128. Как хранить audit log?

Ожидаемый ответ: audit log должен быть append-only, содержать actor, action, target, timestamp, correlation/request id, before/after или diff, source/service, metadata. Для критичных изменений audit пишется в той же транзакции, что и бизнес-изменение.

Объяснение: audit нужен для расследований и compliance, поэтому он должен быть надежным и защищенным от тихого изменения. Но нельзя бездумно писать PII/secrets; нужны retention и access control.

129. Что такое outbox pattern и почему он связан с БД?

Ожидаемый ответ: outbox pattern решает проблему атомарности между записью в БД и публикацией сообщения. В одной транзакции с бизнес-изменением приложение записывает outbox row. Отдельный publisher читает outbox и публикует в брокер, затем помечает как отправленное.

Объяснение: без outbox возможны два сбоя: БД commit прошел, событие не отправлено; или событие отправлено, а транзакция rollback. Outbox переводит задачу в надежный retryable процесс.

Сильный ответ: consumers все равно должны быть idempotent, потому что publisher может отправить событие повторно.

130. Что такое inbox/idempotent consumer pattern?

Ожидаемый ответ: consumer сохраняет id обработанного сообщения или idempotency key в БД с unique constraint. При повторной доставке он видит, что сообщение уже обработано, и не выполняет side effect повторно.

Объяснение: брокеры часто дают at-least-once delivery. Это значит, что дубли - нормальное поведение, а не исключение. Идемпотентность consumer-а защищает от повторных списаний, повторных email и дублирующих записей.

131. Как проектировать уникальность с учетом race conditions?

Ожидаемый ответ: уникальность должна быть закреплена unique constraint/index в БД. Проверка "сначала SELECT, потом INSERT" без constraint не защищает от гонки.

Пример: два запроса регистрации одновременно проверяют, что email свободен, оба видят пусто и оба пытаются вставить. Только unique index гарантирует, что один insert пройдет, второй получит conflict.

Практика: application-level pre-check может улучшить UX, но source of truth - constraint. Ошибку unique violation нужно мапить в доменную ошибку.

132. Как хранить деньги и decimal-значения?

Ожидаемый ответ: не использовать float для денег из-за ошибок бинарного представления и округления. Использовать integer minor units (cents) или decimal/numeric тип с явными правилами precision/scale.

Объяснение: финансовые операции требуют детерминированного округления и точности. В Go можно использовать int64 minor units или decimal library, но важно не смешивать валюты и хранить currency code.

133. Как моделировать many-to-many связь?

Ожидаемый ответ: через join table с foreign keys, например user_roles(user_id, role_id), unique constraint на пару, индексы под оба направления запросов. Если у связи есть атрибуты, join table становится полноценной сущностью.

Объяснение: array/json со списком ids может казаться проще, но ломает referential integrity, усложняет запросы, индексы и каскадные изменения.

134. Когда JSON/JSONB в реляционной БД - хорошая идея, а когда плохая?

Ожидаемый ответ: JSONB полезен для редких, гибких, слабо структурированных attributes, metadata, внешних payloads, когда не все поля участвуют в строгих связях. Плохая идея хранить в JSON ключевые доменные поля, по которым нужны constraints, joins, частые фильтры и аналитика.

Объяснение: JSONB снижает миграционную стоимость, но переносит часть схемы в приложение. Это может ухудшить качество данных и усложнить индексацию.

135. ORM vs raw SQL: какие trade-off?

Ожидаемый ответ: ORM ускоряет типовые CRUD, mapping и миграцию моделей, но может скрывать SQL, создавать N+1, сложные неэффективные запросы и привязывать доменную модель к persistence. Raw SQL прозрачен и оптимизируем, но требует больше boilerplate.

Сильный ответ: выбор не религиозный. Для сложных запросов и performance-critical paths часто лучше явный SQL, для простых CRUD может быть ORM/query builder. Важно логировать SQL и тестировать реальные queries.

136. Как диагностировать медленный endpoint: БД или приложение?

Ожидаемый ответ: смотреть distributed tracing или хотя бы latency breakdown: время handler-a, время SQL, ожидание connection pool, внешние API, serialization. В БД смотреть slow query log, EXPLAIN ANALYZE, locks, CPU/IO, cache hit ratio.

Объяснение: endpoint может быть медленным не из-за самого SQL, а из-за ожидания свободного connection, lock wait, N+1 queries, большого JSON response, GC pressure или внешнего API.

137. Что такое lock wait и как его увидеть?

Ожидаемый ответ: lock wait - время ожидания блокировки, которую держит другая транзакция. Его можно увидеть через системные views БД, slow query logs, wait events, monitoring. В PostgreSQL смотрят pg_stat_activity, pg_locks, wait_event.

Объяснение: запрос может быть простым по плану, но висеть из-за блокировки. Поэтому одного EXPLAIN недостаточно для live-инцидента; нужно смотреть активные транзакции и locks.

138. Почему долгие транзакции опасны даже без активной работы?

Ожидаемый ответ: долгая транзакция держит snapshot, может удерживать locks, мешать vacuum/cleanup старых версий строк, увеличивать bloat, занимать connection и блокировать schema changes.

Объяснение: в MVCC БД старые версии строк нельзя очистить, пока они потенциально видимы открытой транзакции. Поэтому "idle in transaction" - серьезная production-проблема.

139. Что такое MVCC?

Ожидаемый ответ: Multi-Version Concurrency Control позволяет транзакциям видеть согласованный snapshot данных, пока другие транзакции пишут новые версии строк. Читатели и писатели меньше блокируют друг друга, но появляются старые версии, vacuum/cleanup и особенности изоляции.

Объяснение: MVCC объясняет, почему read committed видит новые committed данные между statement-ами, почему долгие транзакции мешают cleanup, и почему update фактически создает новую версию строки.

140. Что такое vacuum/bloat в PostgreSQL?

Ожидаемый ответ: update/delete создают dead tuples. Vacuum очищает их, когда они больше не видимы активным транзакциям. Bloat - разрастание таблиц/индексов из-за накопления неиспользуемого пространства.

Объяснение: если autovacuum не успевает или долгие транзакции удерживают старые версии, таблица растёт, запросы читают больше страниц, cache хуже работает. Это влияет на latency и storage.

141. Как проектировать retention и удаление большого объема данных?

Ожидаемый ответ: не удалять миллионы строк одним большим DELETE в рабочее время. Использовать partitioning и drop partition, batch deletes с лимитами, throttling, архивирование, monitoring replication lag/WAL/locks.

Объяснение: массовое удаление создает locks, WAL, bloat, нагрузку на replica и vacuum. Для time-series данных partition drop часто намного безопаснее.

142. Что такое cache-aside и какие риски у кеширования данных из БД?

Ожидаемый ответ: cache-aside: приложение сначала читает cache, при miss читает БД и кладет значение в cache. Риски: stale data, cache stampede, invalidation complexity, inconsistent serialization, hot keys.

Объяснение: cache не исправляет плохую модель данных автоматически. Он добавляет consistency trade-off. Нужны TTL, jitter, singleflight/request coalescing, invalidation strategy и metrics hit rate.

143. Что такое cache stampede?

Ожидаемый ответ: cache stampede возникает, когда популярный ключ истекает, и много запросов одновременно идут в БД пересчитывать одно значение.

Решения: TTL jitter, early refresh, singleflight/locking, stale-while-revalidate, background refresh, request collapsing. Для критичных hot keys можно прогреть cache заранее.

144. Как выбрать между нормализацией и денормализацией?

Ожидаемый ответ: нормализация снижает дублирование и аномалии записи, улучшает целостность. Денормализация ускоряет чтение и упрощает read models, но создает риск

рассинхронизации и усложняет write path.

Объяснение: правильный выбор зависит от read/write ratio, требований консистентности и сложности запросов. Часто базовая transactional model нормализована, а денормализованные read models строятся асинхронно.

145. Что такое referential integrity и когда нужны foreign keys?

Ожидаемый ответ: foreign keys гарантируют, что ссылка указывает на существующую запись, и задают поведение при delete/update. Они защищают качество данных на уровне БД.

Trade-off: FK добавляют проверки и могут усложнять bulk loads/sharding, но отказ от FK требует надежной компенсации в приложении и monitoring orphan records. Для большинства обычных OLTP-систем FK полезны.

146. Что такое ON DELETE CASCADE и какие риски?

Ожидаемый ответ: ON DELETE CASCADE автоматически удаляет дочерние строки при удалении родителя. Риск - случайное удаление большого дерева данных, долгие locks, неожиданная потеря audit/history.

Практика: использовать осознанно, ограничивать права, иметь soft delete/audit там, где данные критичны, и понимать размер каскада.

147. Как хранить и проверять права доступа в БД-запросах?

Ожидаемый ответ: права лучше учитывать прямо в запросах через tenant_id/user_id/ACL joins, чтобы не загружать лишние данные и не отфильтровывать их только в памяти. В multi-tenant системе почти все таблицы должны иметь tenant boundary, если нет отдельной физической изоляции.

Объяснение: фильтрация доступа после чтения может привести к утечке данных через баг, logs, cache или side channel. DB-level Row Level Security может помочь, но тоже требует дисциплины и тестов.

148. Какие вопросы задать кандидату по реальному инциденту с БД?

Ожидаемый ответ: попросить описать симптомы, метрики, запросы, locks, connection pool, изменения перед инцидентом, mitigation и postmortem. Сильный кандидат говорит о slow query, bad migration, missing index, lock contention, replication lag, pool exhaustion или vacuum/bloat с конкретным расследованием.

Что слушать: не только "добавили индекс", а как убедились, что причина именно в этом; как безопасно применяли fix; какие alerts/tests/runbooks добавили после.

149. Практический кейс: endpoint GET /orders стал медленным после роста данных. Как расследовать?

Ожидаемый ответ: начать с воспроизведения и метрик: latency percentiles, RPS, error rate, DB time, number of queries per request. Проверить N+1, slow query log, EXPLAIN ANALYZE, наличие индексов под filters/sort, offset pagination, размер response, connection pool wait.

Объяснение: рост данных часто проявляет сразу несколько проблем: offset стал большим, индекс не покрывает WHERE tenant_id AND status ORDER BY created_at, ORM делает N+1 для items, response сериализует слишком много полей.

Хорошее решение: добавить/исправить composite или partial index, перейти на cursor pagination, batch-load related data, ограничить page size, добавить metrics и regression test/benchmark для query plan.

150. Практический кейс: иногда создание пользователя возвращает 500 вместо "email занят". Что проверить?

Ожидаемый ответ: проверить unique constraint на email, обработку unique violation в Go, race между pre-check и insert, нормализацию email, collation/case sensitivity, transaction boundaries.

Объяснение: корректная схема: нормализовать email или использовать case-insensitive type/index, иметь unique constraint, при конфликте мапить DB error в доменную ErrEmailAlreadyExists и HTTP 409. Предварительный SELECT не заменяет constraint.

151. Практический кейс: воркеры периодически берут одну и ту же job дважды. Как исправить?

Ожидаемый ответ: использовать атомарное claim-обновление или locking pattern: `SELECT ... FOR UPDATE SKIP LOCKED` внутри транзакции, затем `update status`; либо `UPDATE ... WHERE status='pending' ... RETURNING`. Нужны lease/visibility timeout для recovery после падения worker-a.

Объяснение: простая схема "SELECT pending, потом UPDATE" без lock/condition race-prone. Два worker-a могут выбрать одну строку до изменения статуса.

Сильный ответ: кандидат учитывает idempotency обработки job, retry count, next_run_at, dead-letter status и мониторинг queue lag.

152. Практический кейс: после релиза выросло число DB connections. Что делать?

Ожидаемый ответ: проверить настройки pool в сервисе, количество pod-ов, утечки rows (`rows.Close()`), долгие транзакции, новые background workers, retry loops, health checks, connection lifetime. Сравнить DB max connections и суммарный потенциальный pool всех инстансов.

Объяснение: в Go `database/sql` lazy-open, но при нагрузке может открыть до `MaxOpenConns` на каждый процесс. Если лимит не задан, поведение может перегрузить БД.

153. Практический кейс: миграция зависла и заблокировала записи. Как действовать?

Ожидаемый ответ: сначала стабилизировать: понять blocking query/lock, оценить impact, при необходимости cancel/terminate migration, откатить deploy или отключить traffic. Потом разобрать DDL, lock level, размер таблицы, наличие long transactions.

Профилактика: lock timeout для миграций, statement timeout, online/concurrent operations, dry run на staging с похожим объемом, миграции маленькими шагами, мониторинг locks и replication lag.

154. Какие DB-вопросы стоит считать обязательными для Middle+ Go backend?

Ожидаемый ответ: транзакции и rollback/commit, базовые индексы и EXPLAIN, N+1, connection pool, уникальность через constraints, migration safety, isolation/lost update, context timeouts для SQL, обработка DB errors.

Объяснение: Middle+ не обязан быть DBA, но должен писать код, который не ломает данные и не создает очевидных production-проблем. Если кандидат не понимает транзакции, индексы и конкурентные обновления, это серьезный риск для backend-роли.

17.14 Как оценивать ответы

Уровень ниже Middle+

- Знает термины, но не может объяснить последствия в production.
- Предлагает unbounded concurrency, retries без timeout, глобальные переменные и panic для обычных ошибок.
- Путается в context, interfaces, transactions, race conditions.
- Не упоминает тесты, observability и edge cases без подсказки.

Уверенный Middle+

- Объясняет базовые механики Go и типичные ловушки.
- Самостоятельно проектирует bounded concurrency и корректно использует context.
- Понимает транзакции, индексы, миграции и надежную работу с внешними API.
- Пишет тестируемый код и видит эксплуатационные риски.
- Может аргументировать trade-off и коммуницировать спокойно.

Сильный Middle+ / Senior-

- Начинает с требований и ограничений, а не с любимой технологии.
- Видит failure modes: partial failure, duplicate delivery, race, deadlock, overload.
- Предлагает простое решение с понятным путем развития.
- Связывает код, архитектуру и эксплуатацию: metrics, alerts, rollout, incident response.
- Умеет объяснить, какие решения являются обязательными сейчас, а какие можно отложить.